

Fundamentos de Computadores

1º curso del Grado en Ingeniería Informática

Seminario

Introducción al uso del software Vivado de Xilinx



*Departamento de Ingeniería Electrónica,
de Sistemas Informáticos y Automática*

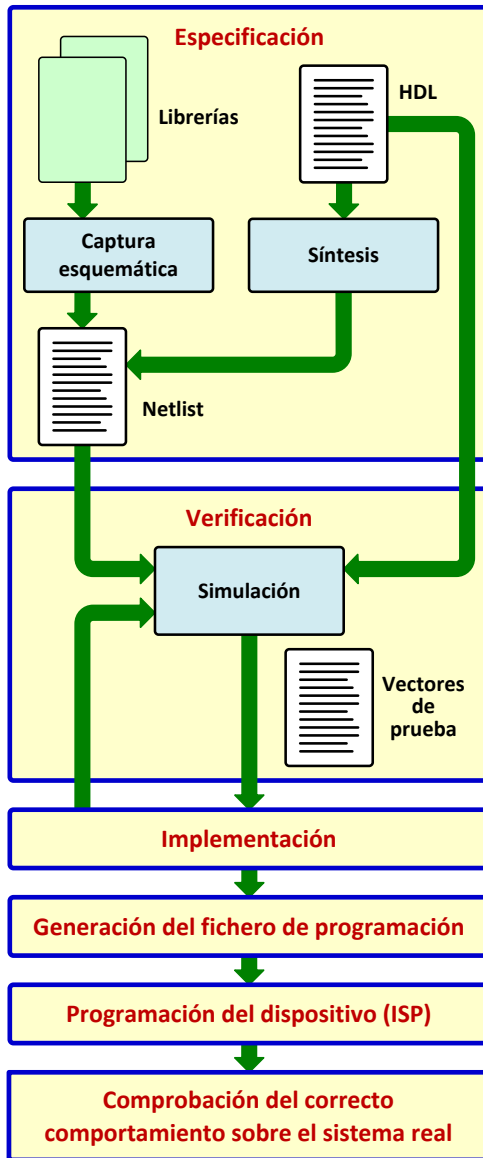
Seminario: Introducción al uso del software Vivado de Xilinx

Contenido del seminario

- Flujo de diseño con PLDs.
- La tarjeta de desarrollo ZedBoard de Digilent.
- Breve descripción de las FPGAs.
- El entorno de trabajo Vivado.
- Ejecución del software vivado.
- Creación de un proyecto.
- Edición del fichero fuente.
- Simulación del sistema.
- Síntesis del sistema.
- Implementación del sistema.
- Programación de la tarjeta de desarrollo.

Seminario: Introducción al uso del software Vivado de Xilinx

Flujo de diseño con PLDs

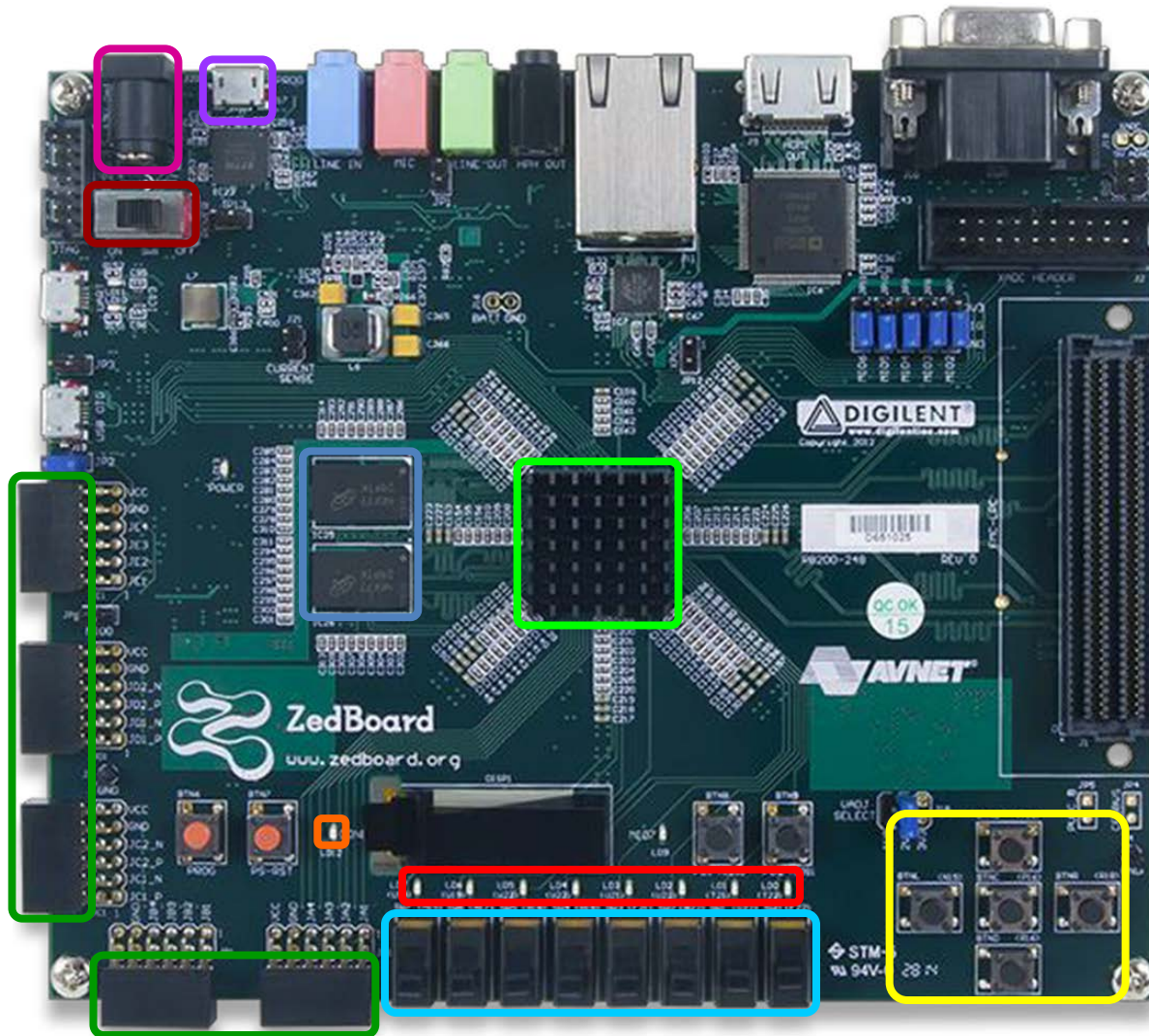


Programación en sistema (ISP)

Seminario: Introducción al uso del software Vivado de Xilinx

La tarjeta de desarrollo ZedBoard de Digilent

- En el presente seminario se explica el modo de implementar sistemas digitales mediante la tarjeta de desarrollo **ZedBoard** de Digilent, cuya vista superior se muestra en la figura.



Conector de alimentación

Interruptor de encendido

Conector USB de programación

FPGA (XC7Z020-CLG484-1)

Memorias DDR3

LED "DONE"

Pulsadores de usuario

LEDs de usuario

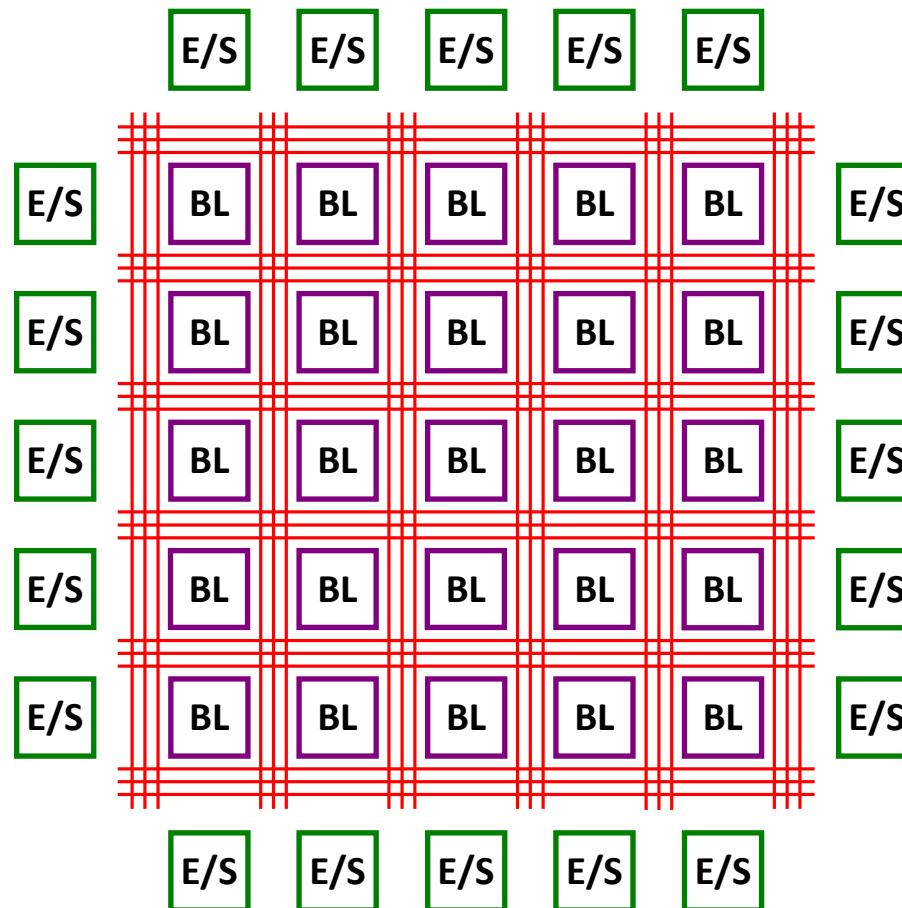
Conmutadores de usuario

Conectores de expansión Pmod

Seminario: Introducción al uso del software Vivado de Xilinx

Breve descripción de las FPFAs

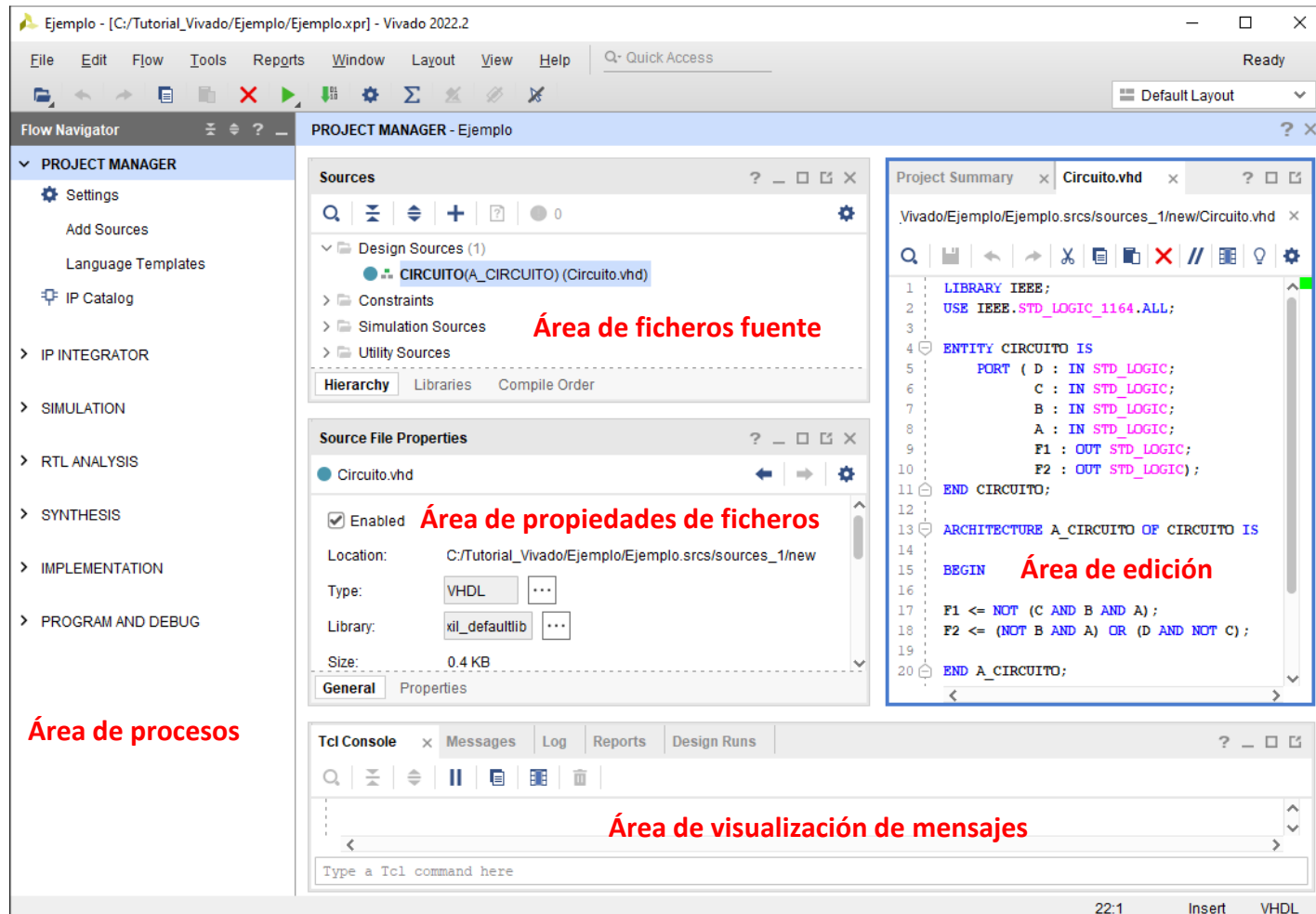
- Las FPGAs (**Field Programmable Gate Arrays**) son estructuras programables formadas por tres elementos básicos: bloques lógicos internos (BL), bloques lógicos de entrada/salida (E/S) y recursos de interconexión.



Seminario: Introducción al uso del software Vivado de Xilinx

El entorno de trabajo Vivado

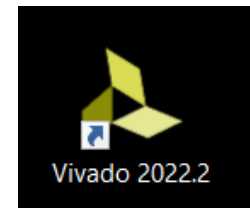
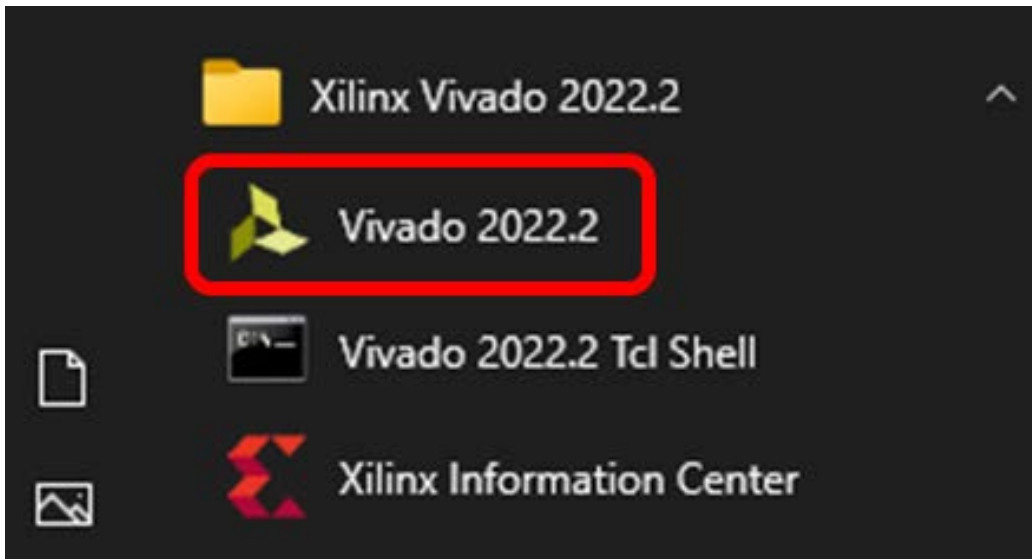
- Para llevar a cabo las diferentes tareas que implica el proceso de diseño con FPGAs, en este tutorial se emplea el software **Vivado** de **Xilinx**. En la figura se resaltan las principales áreas de la pantalla principal del entorno de trabajo de este programa.



Seminario: Introducción al uso del software Vivado de Xilinx

Ejecución del software Vivado

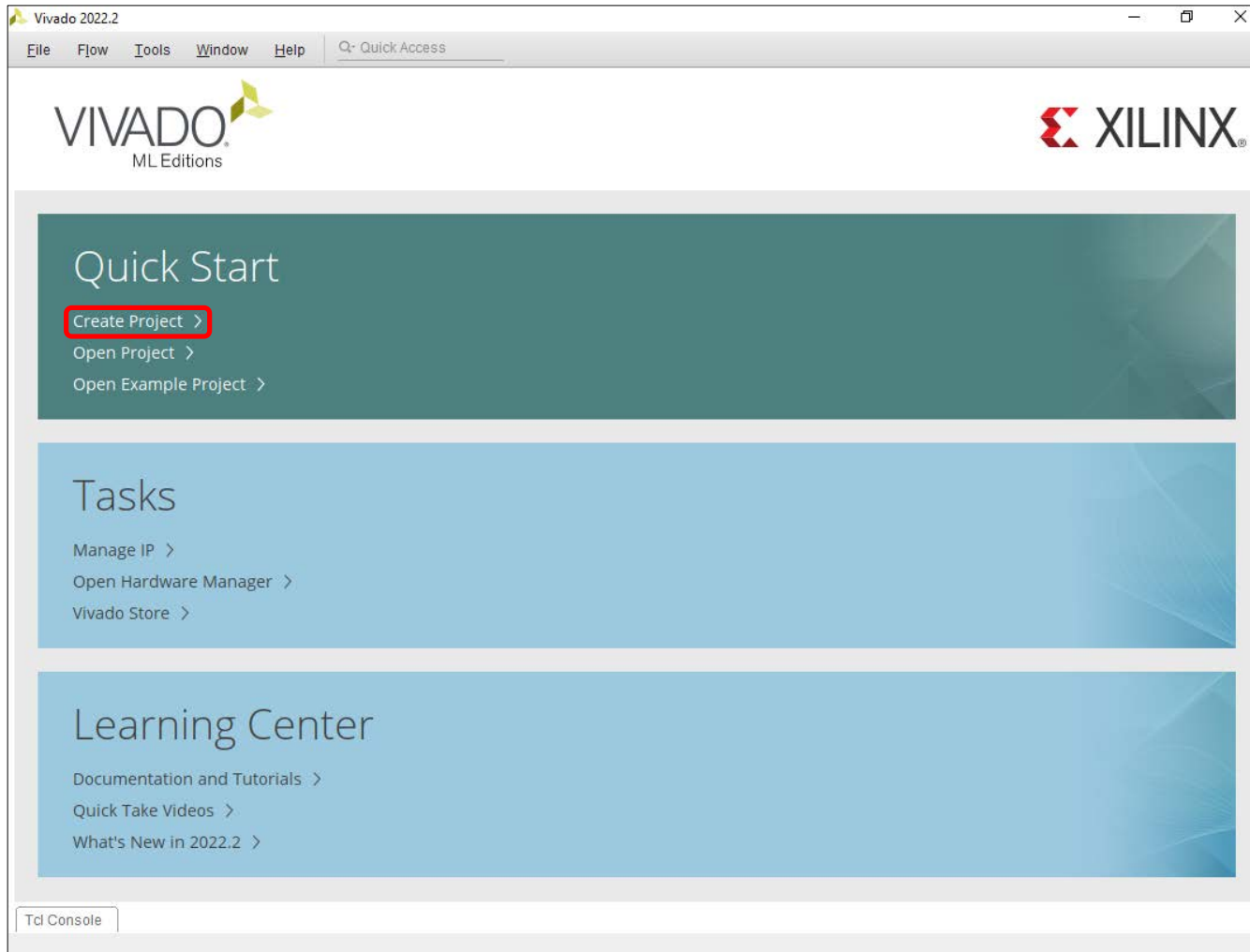
- Antes de proceder a la creación de un proyecto ejecutamos el software Vivado, bien desde el menú de aplicaciones de Windows, o bien haciendo click sobre un acceso directo colocado en el escritorio.



Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

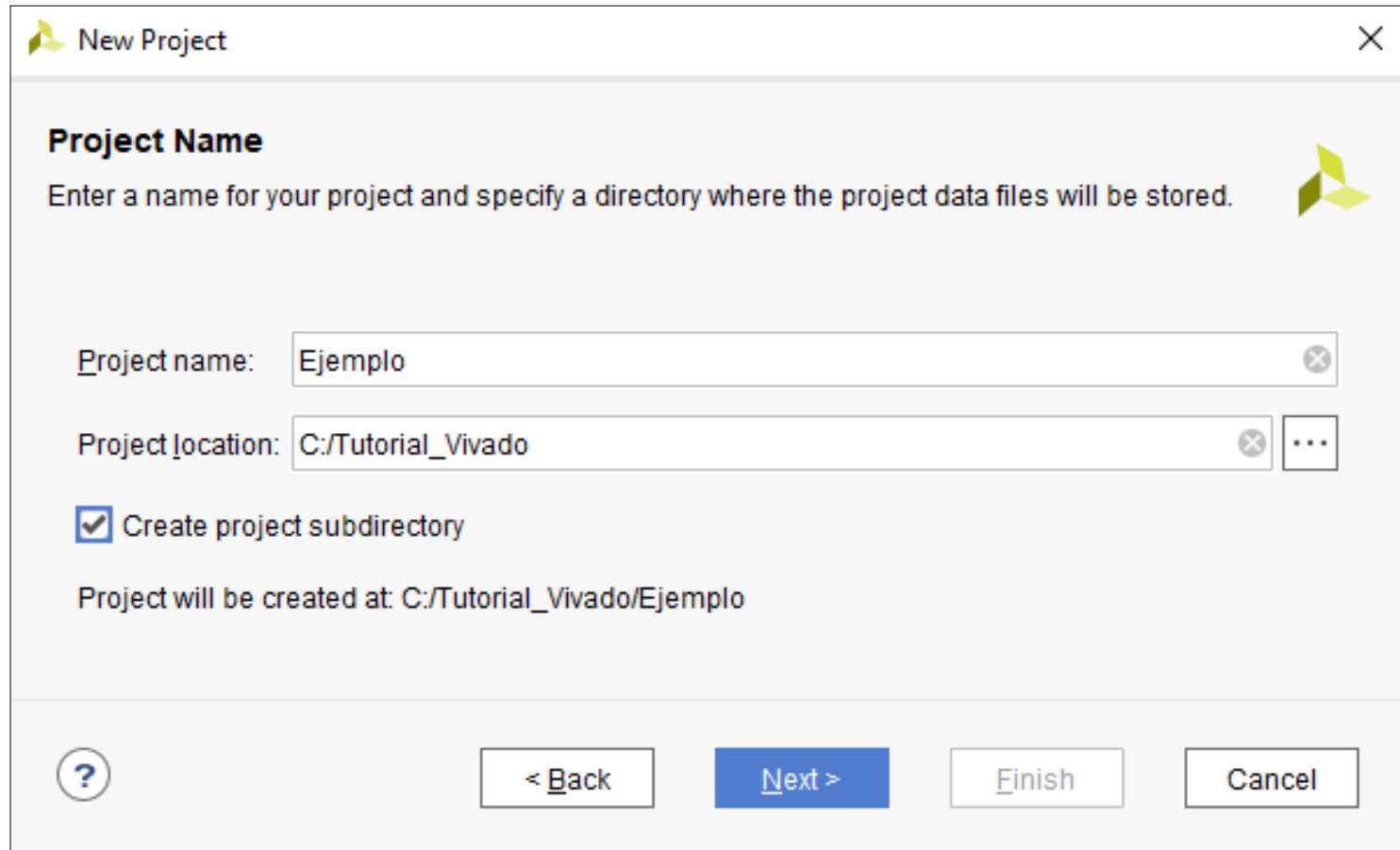
- Tras la ejecución del software Vivado aparece una ventana como la mostrada en la figura, en la que seleccionamos **Create Project** para iniciar la creación de un nuevo proyecto.



Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

- Tras la ejecución de **Create Project** y posterior confirmación mediante **Next** aparece la ventana **New Project**, donde especificamos el nombre a asignar al proyecto y la ubicación donde éste se almacenará. Debemos activar la opción **Create project subdirectory**.



New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name:

Project location:

☒ Create project subdirectory

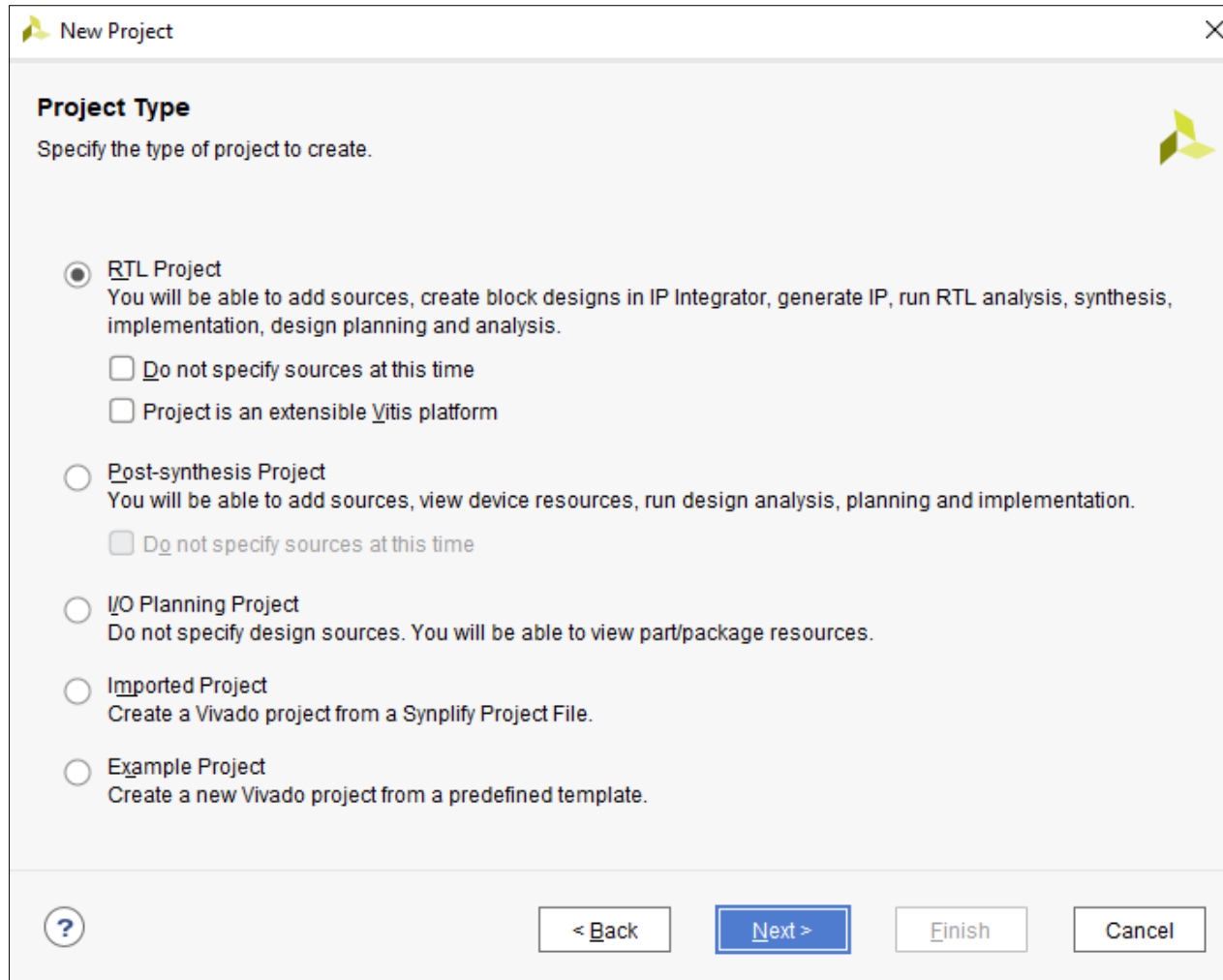
Project will be created at: C:/Tutorial_Vivado/Ejemplo



Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

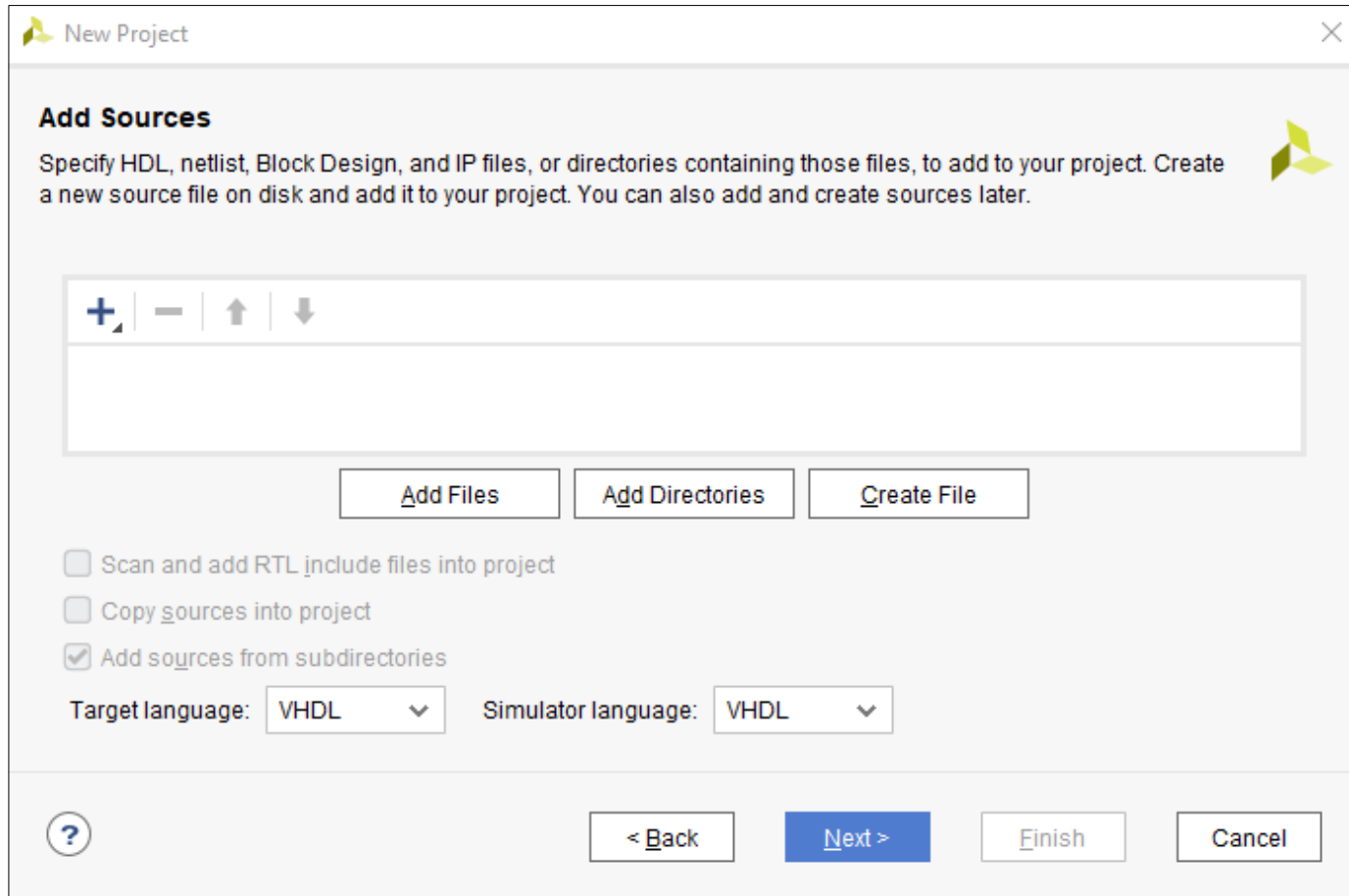
- Tras validar mediante **Next** aparece una nueva ventana en la que seleccionamos la opción **RTL Project** y seguidamente hacemos click sobre **Next**.



Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

- En la siguiente ventana podemos añadir al proyecto ficheros fuente ya existentes, mediante la opción **Add Files**, o crear nuevos archivos, picando sobre **Create File**. Hacemos esto último y validamos con **Next**, pero previamente, tanto en **Target language** como en **Simulator language**, debemos seleccionar **VHDL**.



New Project

Add Sources

Specify HDL, netlist, Block Design, and IP files, or directories containing those files, to add to your project. Create a new source file on disk and add it to your project. You can also add and create sources later.

+

–

↑

↓

Add Files **Add Directories** **Create File**

☐ Scan and add RTL include files into project

☐ Copy sources into project

☒ Add sources from subdirectories

Target language: **VHDL** Simulator language: **VHDL**

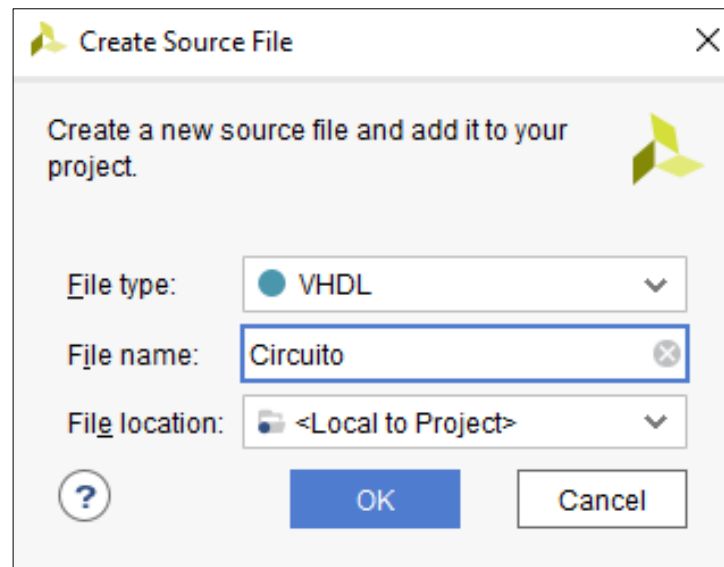
? < Back Next > Finish Cancel

- Para ilustrar este tutorial nos valemos de un ejemplo, consistente en la implementación de un circuito combinacional sencillo compuesto por dos funciones **F1** y **F2** que dependen de cuatro variables de entrada **D**, **C**, **B** y **A**, cuyas expresiones son las siguientes:

$$F_1(D, C, B, A) = \overline{CBA} = \sum_4(0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13, 14)$$

$$F_2(D, C, B, A) = \overline{BA} + \overline{DC} = \sum_4(1, 5, 8, 9, 10, 11, 13)$$

- Tras pulsar sobre **Create File**, aparece la siguiente ventana, donde introducimos el nombre a asignar al fichero fuente VHDL. Introducimos **Circuito** y validamos con **OK**. El nuevo fichero creado se guarda con el nombre **Circuito.vhd**.



Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

- Una vez validado con **OK**, aparece la ventana de la figura, donde se observa que ya aparece el fichero fuente a crear. En la opción **HDL Source For** del fichero fuente, debemos seleccionar **Synthesis & Simulation**, ya que no sólo queremos simular el diseño, sino sintetizarlo. Tras hacer esto validamos con **Next**.

New Project

Add Sources

Specify HDL, netlist, Block Design, and IP files, or directories containing those files, to add to your project. Create a new source file on disk and add it to your project. You can also add and create sources later.

	Index	Name	Library	HDL Source For	Location
●	1	Circuito.vhd	xil_defaultlib	Synthesis & Simulation	<Local to Project>

☐ Scan and add RTL include files into project

☐ Copy sources into project

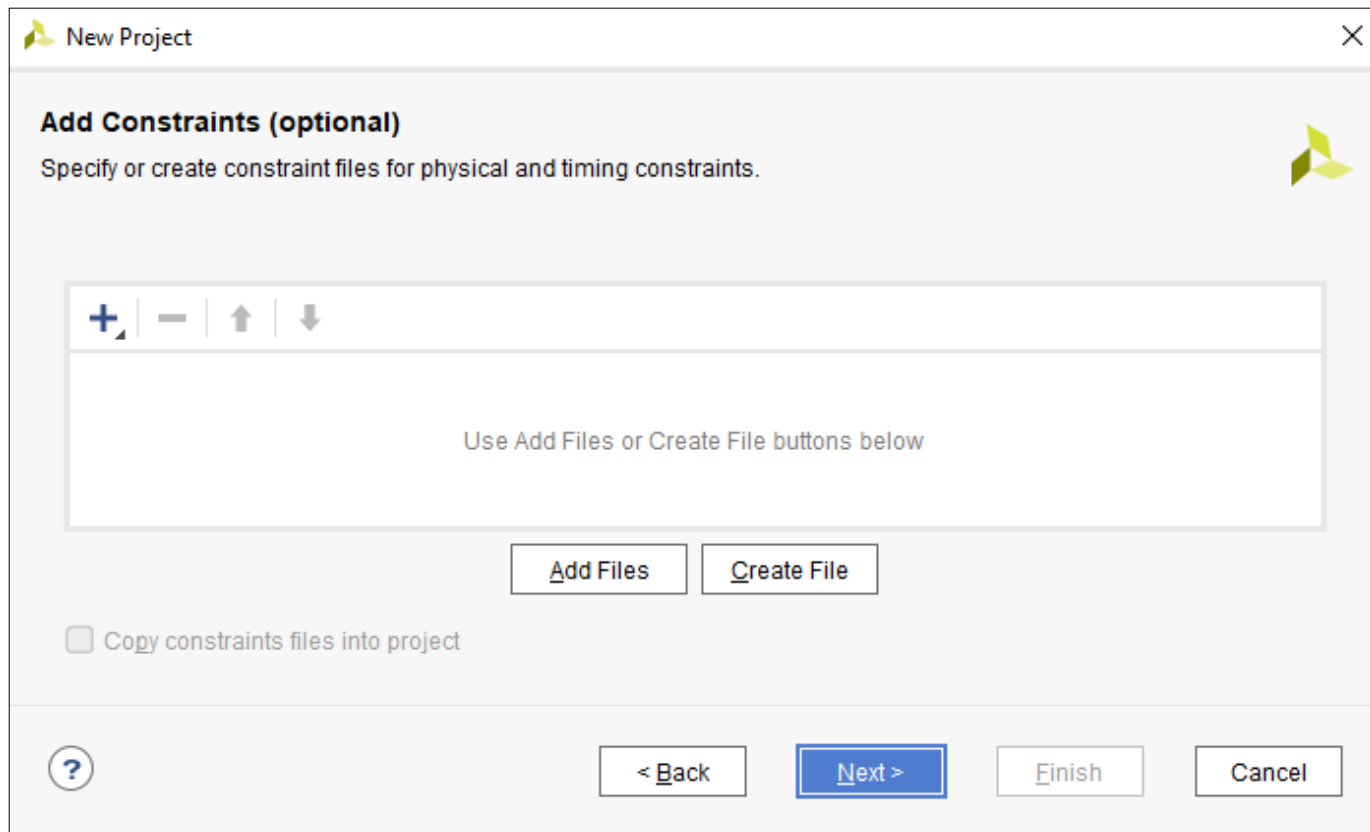
☒ Add sources from subdirectories

Target language: VHDL Simulator language: VHDL

Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

- A continuación, aparece la ventana de la figura, que permite introducir las restricciones del proyecto. De momento, no introducimos restricciones, lo haremos más tarde. Seleccionamos **Next**.



Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

- Al validar con **Next** aparece la ventana **Select Device**, que nos permite configurar las características de la tarjeta de desarrollo en la que se implementará el diseño. Para usar la tarjeta de desarrollo **ZedBoard** configuramos esta ventana como se muestra en la figura.

Filter, search, and browse parts by their resources. The selection will be applied.

Parts | **Boards**

[Reset All Filters](#)

Category: All Package: All Remaining Temperature: All Remaining

Family: Zynq-7000 Speed: All Remaining Static power: All Remaining

Search: Q-

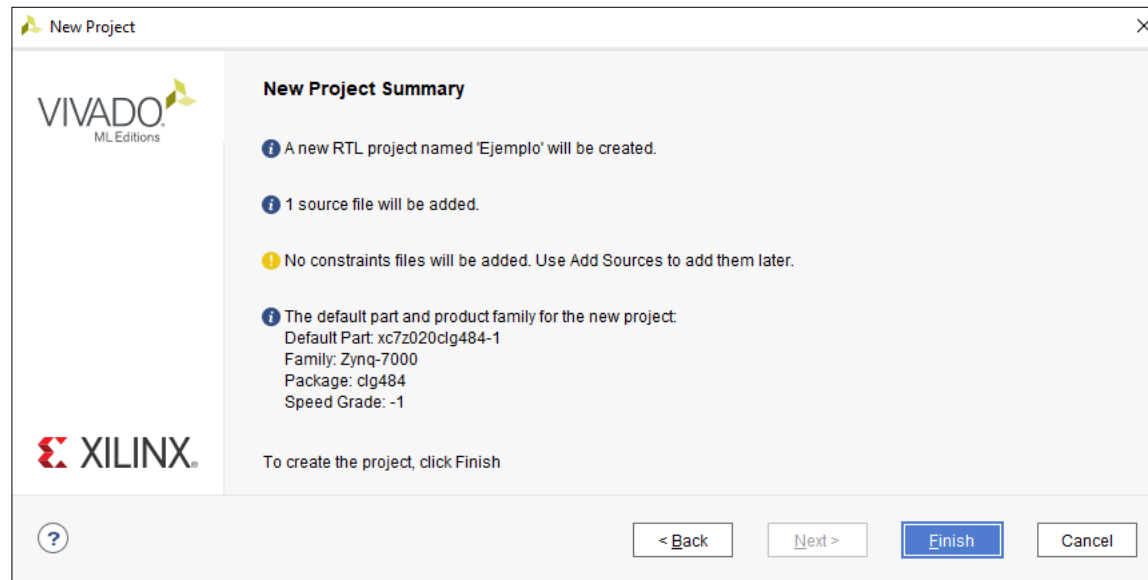
Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs
xc7z020clg484-3	484	200	53200	106400	140	0	220
xc7z020clg484-2	484	200	53200	106400	140	0	220
xc7z020clg484-1	484	200	53200	106400	140	0	220
xc7z020iclg400-1L	400	125	53200	106400	140	0	220
xc7z020iclg484-1L	484	200	53200	106400	140	0	220

OK Cancel

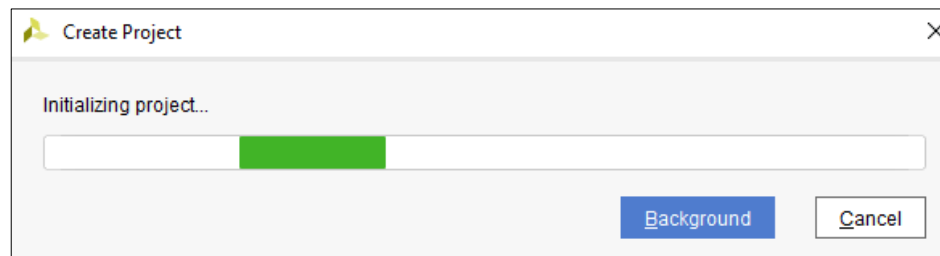
Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

- Una vez configuradas las características de la tarjeta de desarrollo y validadas mediante **Next**, aparece una ventana con las características del proyecto especificadas hasta el momento.



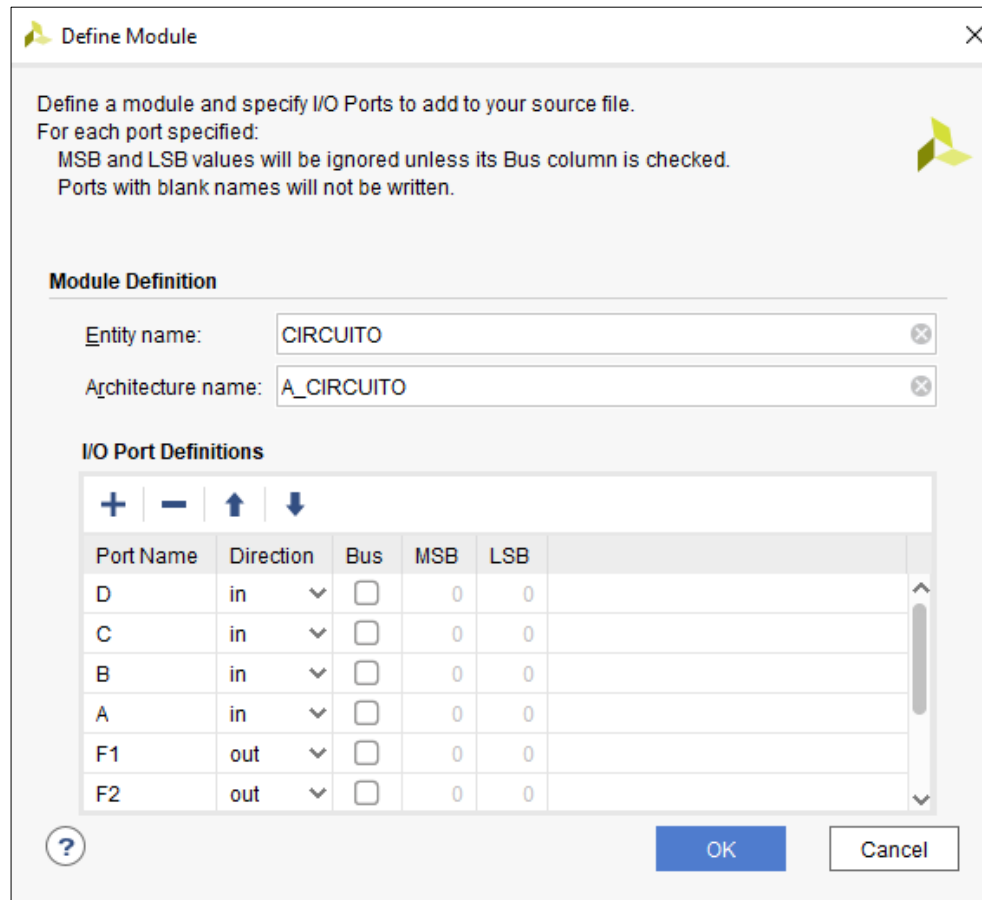
- Hacemos click sobre **Finish** y aparece una ventana que informa del inicio de la creación del proyecto.



Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

- En la siguiente ventana (**Define Module**) se nos pregunta sobre datos concretos del fichero fuente definido anteriormente (**Circuito.vhd**). En ella, asignamos nombres a la entidad y a la arquitectura, y especificamos las características de los terminales de entrada (**D**, **C**, **B** y **A**) y de salida (**F1** y **F2**) del sistema. A continuación, validamos mediante **OK**.



Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

Entity name: CIRCUITO

Architecture name: A_CIRCUITO

I/O Port Definitions

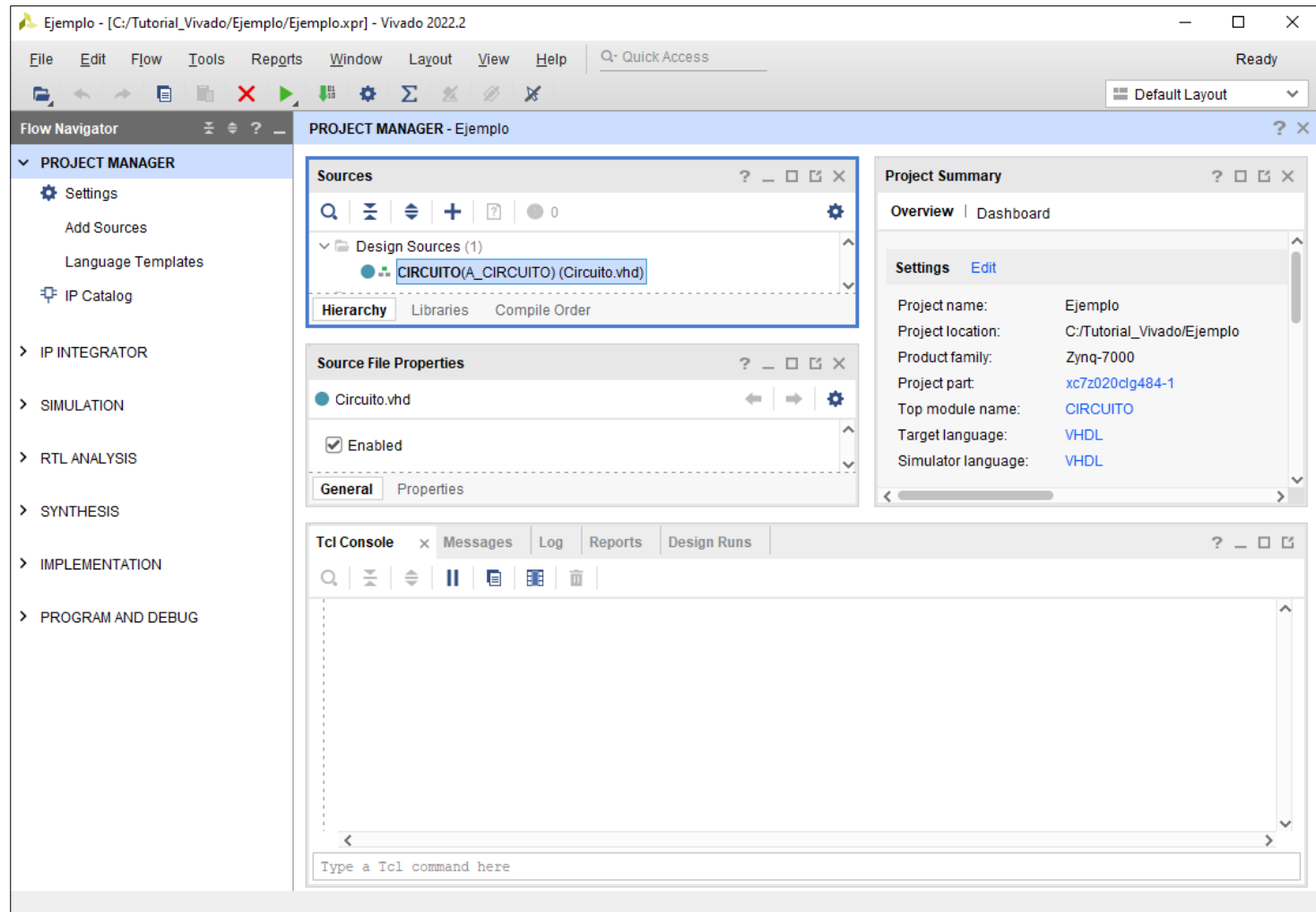
Port Name	Direction	Bus	MSB	LSB
D	in	☐	0	0
C	in	☐	0	0
B	in	☐	0	0
A	in	☐	0	0
F1	out	☐	0	0
F2	out	☐	0	0

Buttons: ? OK Cancel

Seminario: Introducción al uso del software Vivado de Xilinx

Creación de un proyecto

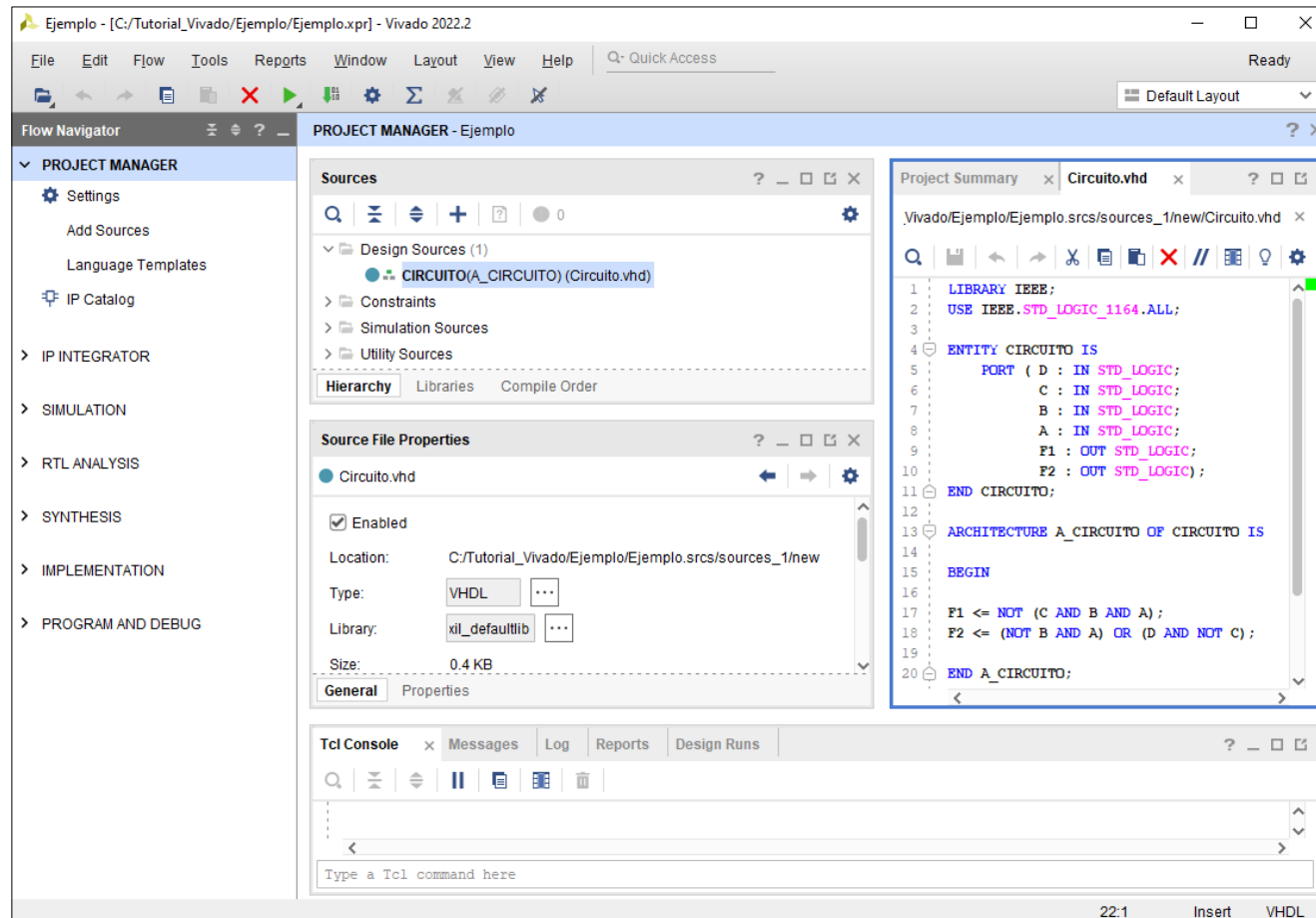
- Tras hacer click sobre **OK** aparece la ventana principal del entorno Vivado, donde en la sección **Sources** ya se muestra el fichero fuente creado previamente (**Circuito.vhd**).



Seminario: Introducción al uso del software Vivado de Xilinx

Edición del fichero fuente

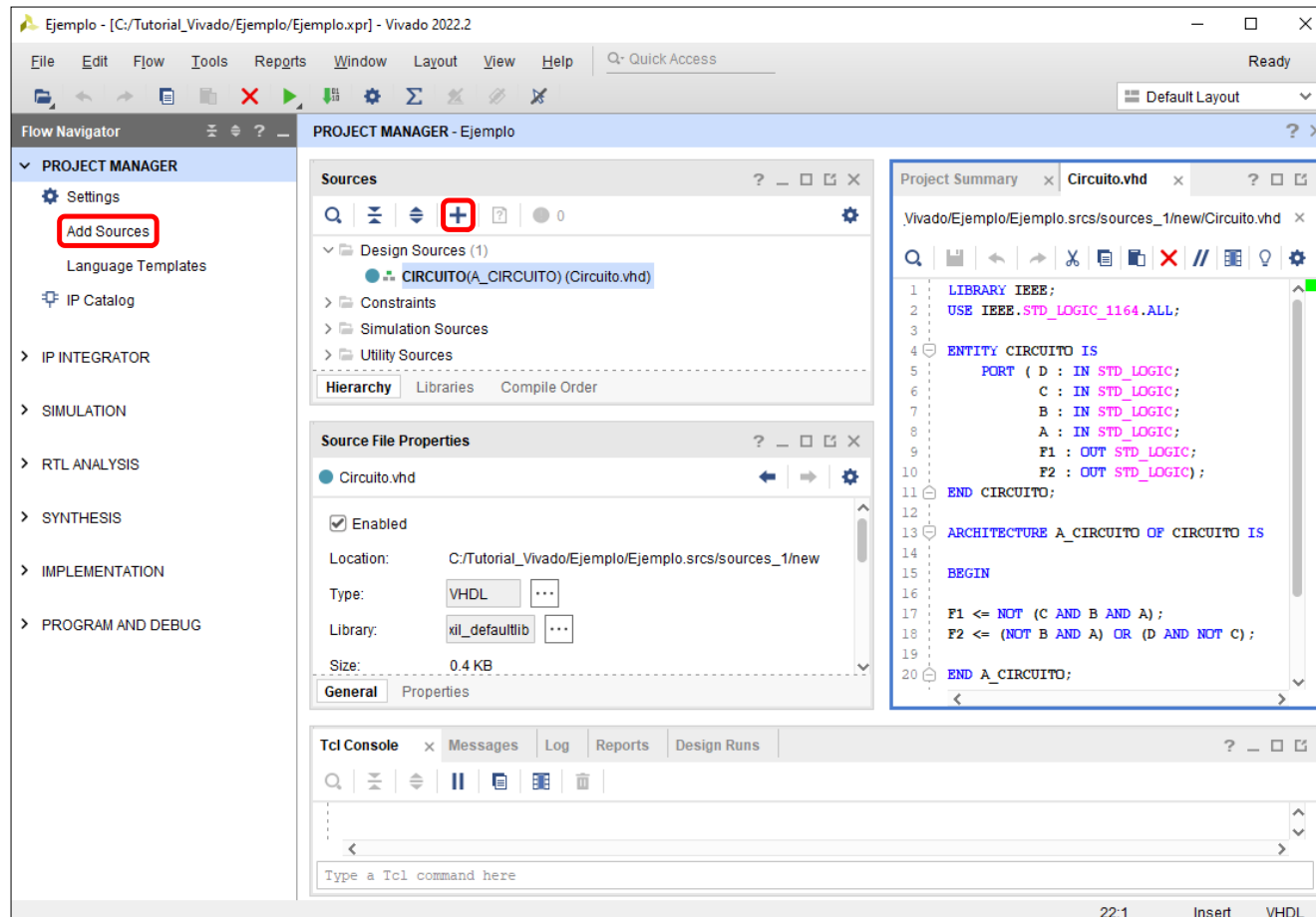
- Una vez creado el proyecto, debemos completar el fichero fuente VHDL. Para ello, hacemos doble clic sobre dicho fichero en la sección **Sources** de la pantalla principal de Vivado, con lo que éste se abre en una ventana ubicada en la parte derecha de la pantalla. En la figura se muestra el contenido del fichero una vez editado para modelar las funciones de salida F1 y F2.



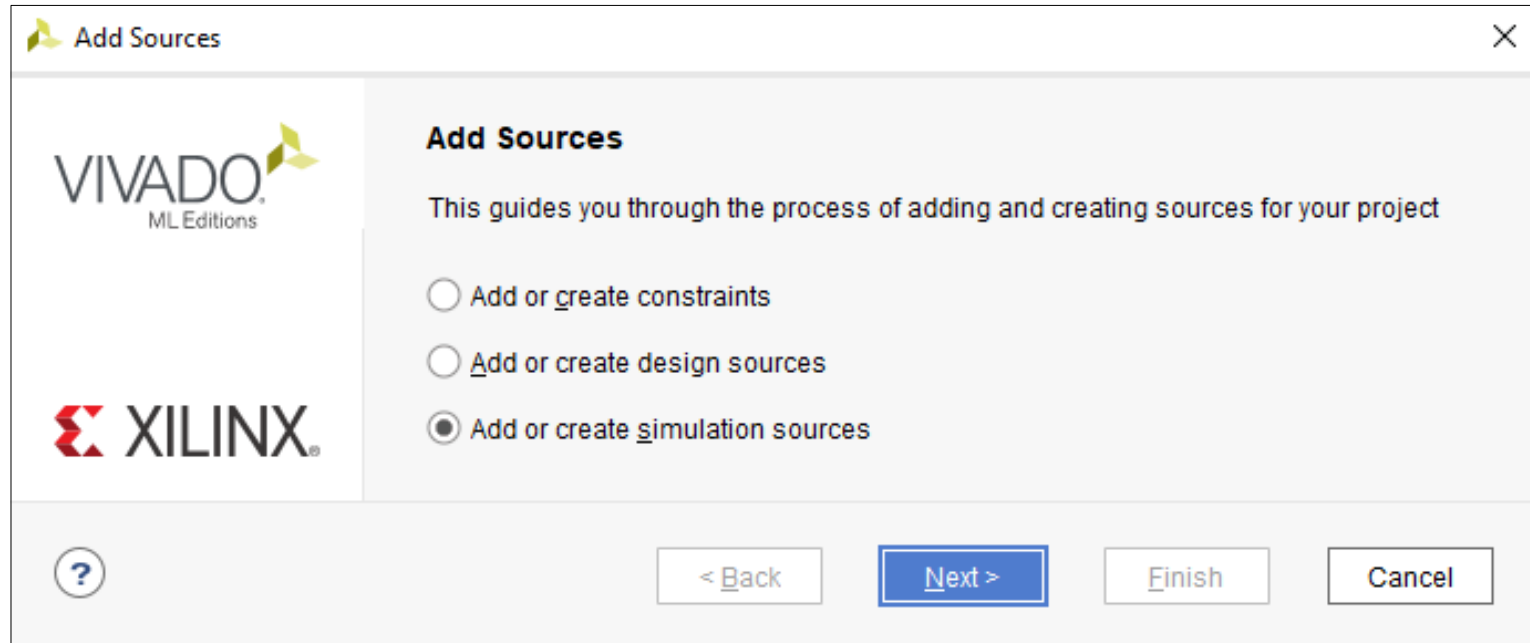
Seminario: Introducción al uso del software Vivado de Xilinx

Simulación del sistema

- Para poder simular el diseño, debemos añadir un nuevo fichero fuente al proyecto de tipo VHDL Test Bench. Para ello, en la zona **Flow Navigator** de la pantalla principal de Vivado, desplegamos **Project Manager** y seleccionamos **Add Sources**, o hacemos click sobre el signo **+** en **Sources**.



- Una vez hecho esto, aparece una ventana emergente donde seleccionamos la opción **Add o créate simulation sources**, como puede apreciarse en la figura.

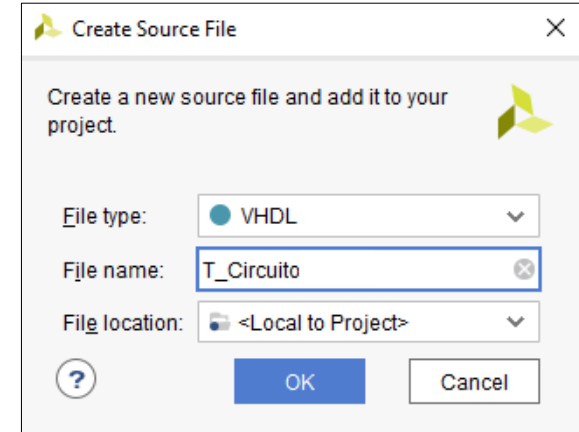
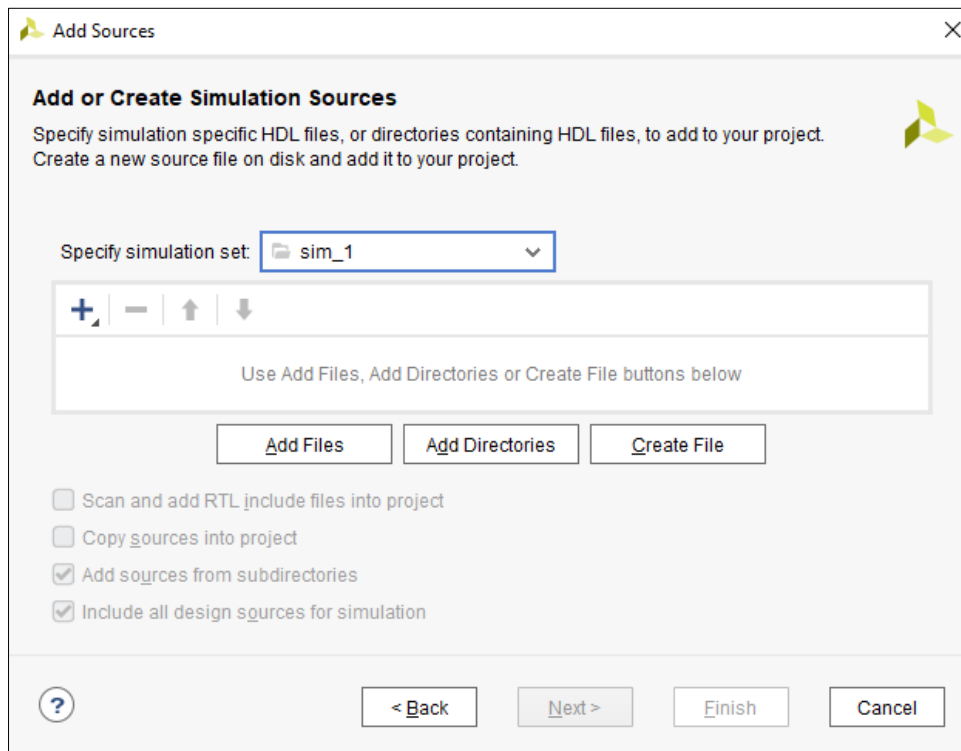


- Una vez seleccionada la opción **Add o créate simulation sources** para el nuevo fichero fuente, validamos mediante la activación de **Next**.

Seminario: Introducción al uso del software Vivado de Xilinx

Simulación del sistema

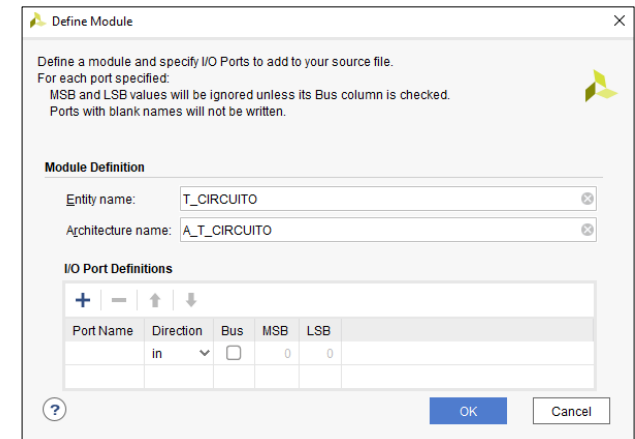
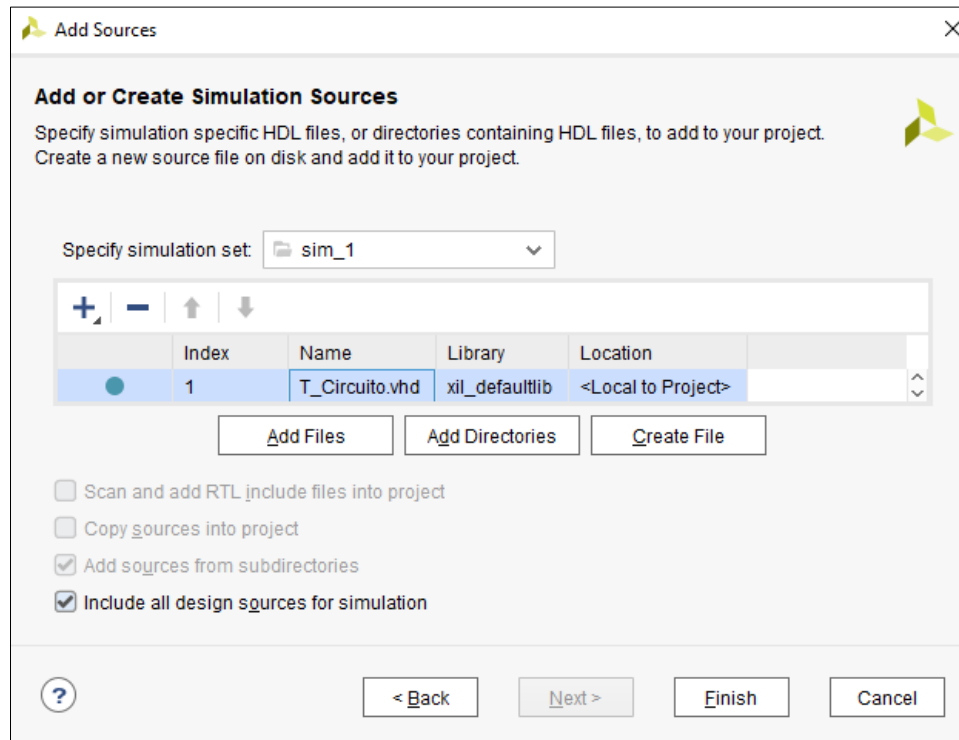
- Tras activar **Next** aparece la ventana de la figura izquierda, mediante la cual podemos añadir al proyecto ficheros fuente de simulación ya existentes, activando la opción **Add Files**, o crear ficheros fuente nuevos haciendo click sobre **Create File**.
- Hacemos esto último, con lo cual aparece una ventana como la de la figura de la derecha, en la que asignamos un nombre al nuevo fichero de simulación a crear (**T_Circuito**).



Seminario: Introducción al uso del software Vivado de Xilinx

Simulación del sistema

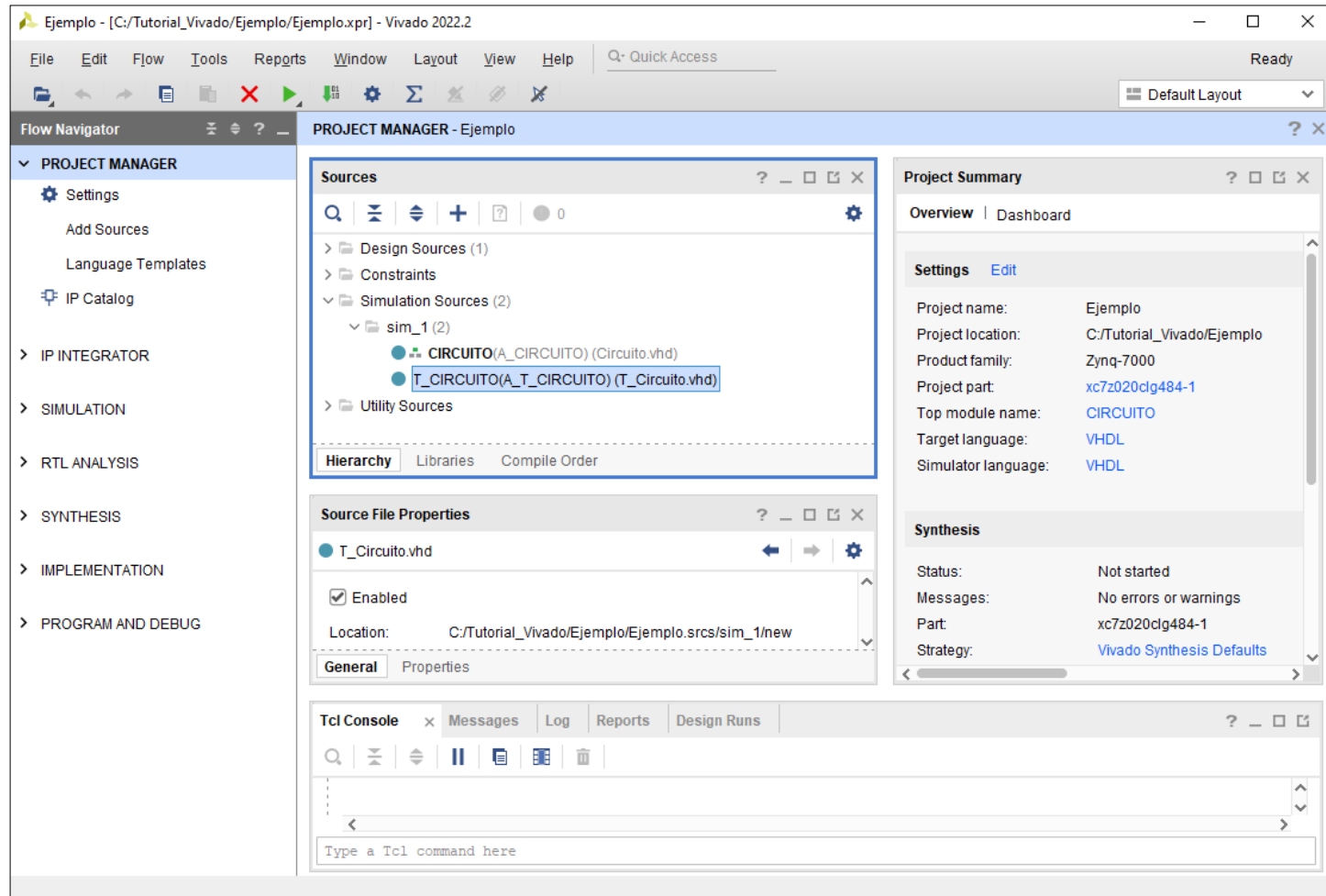
- Una vez introducido el nombre del fichero de simulación y activado **OK**, aparece una ventana como la de la figura izquierda, donde se observa que ya aparece el fichero de simulación a crear.
- Tras pulsar sobre **Finish** se muestra una nueva ventana en la que asignamos nombres tanto a la entidad como a la arquitectura del fichero de simulación. En este caso, la configuramos como se muestra en la figura derecha y a continuación hacemos click sobre **OK**.



Seminario: Introducción al uso del software Vivado de Xilinx

Simulación del sistema

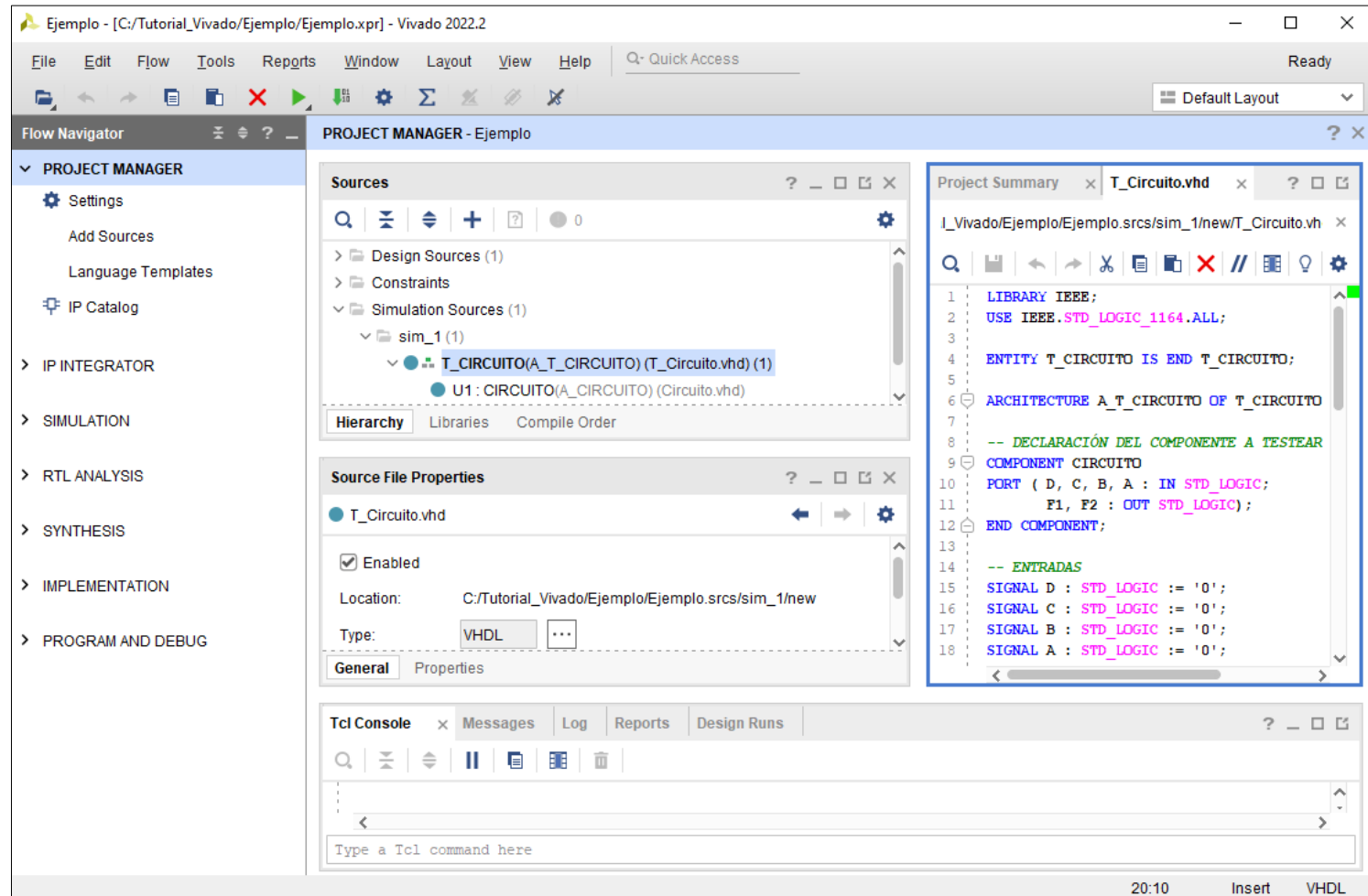
- Una vez validados mediante **OK** los datos del fichero de simulación introducidos, se genera una versión preliminar del mismo, que se muestra en la sección **Sources** de la pantalla principal de Vivado dentro de una carpeta denominada **Simulation Sources**, como se aprecia en la figura.



Seminario: Introducción al uso del software Vivado de Xilinx

Simulación del sistema

- Para editar el fichero de simulación creado previamente, hacemos doble click sobre el nombre de éste (***T_Circuito.vhd***), con lo que el programa abre el fichero en una nueva ventana ubicada en la parte derecha de la pantalla, como se muestra en la figura.



- El contenido del fichero de simulación una vez editado y completado es el siguiente:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY T_CIRCUITO IS END T_CIRCUITO;

ARCHITECTURE A_T_CIRCUITO OF T_CIRCUITO IS

  -- DECLARACIÓN DEL COMPONENTE A TESTEAR (UUT)
  COMPONENT CIRCUITO
    PORT ( D, C, B, A : IN STD_LOGIC;
          F1, F2 : OUT STD_LOGIC);
  END COMPONENT;

  -- ENTRADAS
  SIGNAL D : STD_LOGIC := '0';
  SIGNAL C : STD_LOGIC := '0';
  SIGNAL B : STD_LOGIC := '0';
  SIGNAL A : STD_LOGIC := '0';

  -- SALIDAS
  SIGNAL F1 : STD_LOGIC;
  SIGNAL F2 : STD_LOGIC;

  BEGIN

  -- INSTANCIACIÓN DEL COMPONENTE (UUT)
  U1: CIRCUITO PORT MAP ( A => A,
                          B => B,
                          C => C,
                          D => D,
                          F1 => F1,
                          F2 => F2);
```

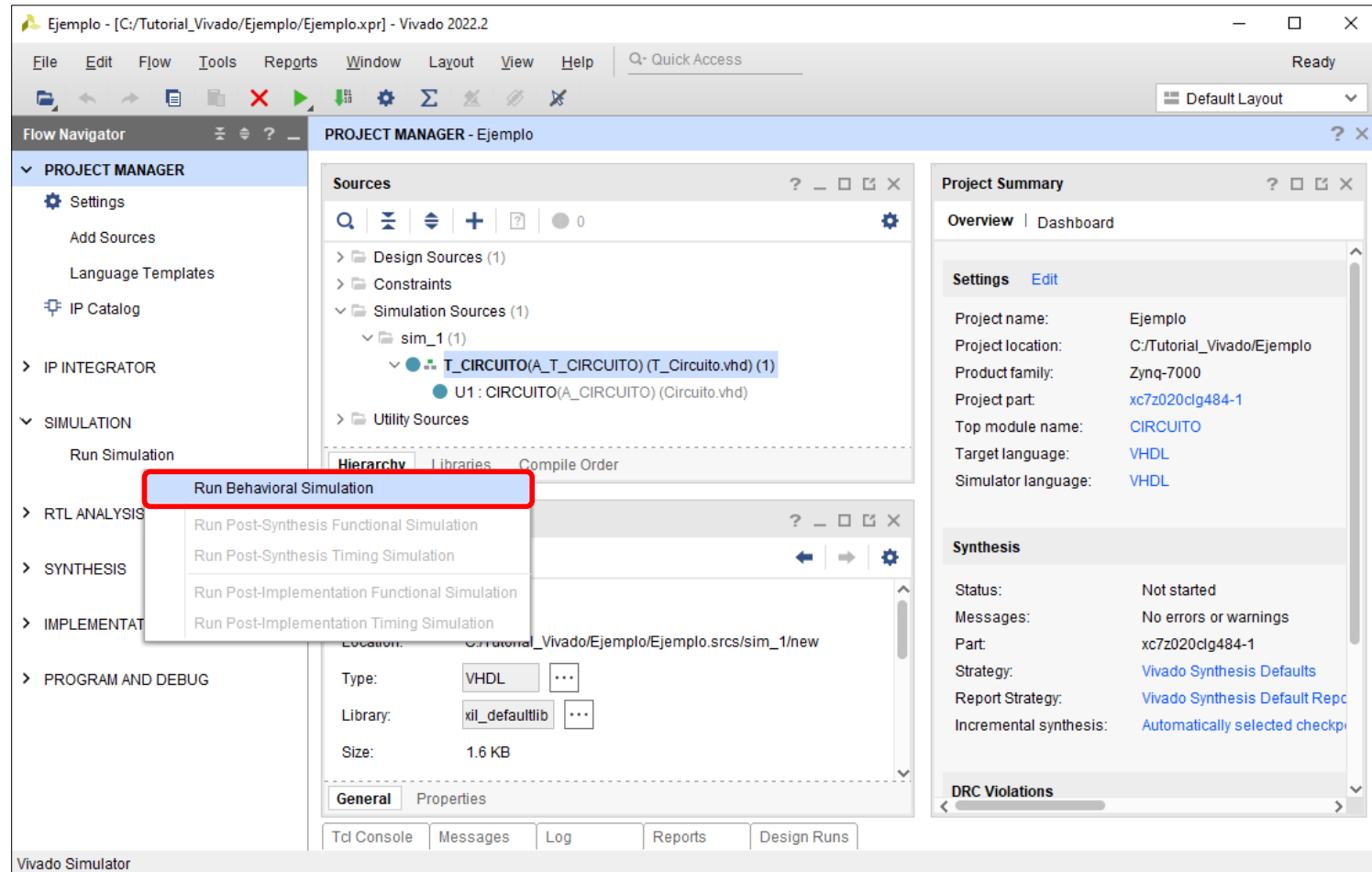
```
-- DEFINICIÓN DE ESTÍMULOS
STIM_PROC: PROCESS
BEGIN
  D <= '0'; C <= '0'; B <= '0'; A <= '0'; WAIT FOR 10 NS;
  D <= '0'; C <= '0'; B <= '0'; A <= '1'; WAIT FOR 10 NS;
  D <= '0'; C <= '0'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
  D <= '0'; C <= '0'; B <= '1'; A <= '1'; WAIT FOR 10 NS;
  D <= '0'; C <= '1'; B <= '0'; A <= '0'; WAIT FOR 10 NS;
  D <= '0'; C <= '1'; B <= '0'; A <= '1'; WAIT FOR 10 NS;
  D <= '0'; C <= '1'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
  D <= '0'; C <= '1'; B <= '1'; A <= '1'; WAIT FOR 10 NS;
  D <= '1'; C <= '0'; B <= '0'; A <= '0'; WAIT FOR 10 NS;
  D <= '1'; C <= '0'; B <= '0'; A <= '1'; WAIT FOR 10 NS;
  D <= '1'; C <= '0'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
  D <= '1'; C <= '0'; B <= '1'; A <= '1'; WAIT FOR 10 NS;
  D <= '1'; C <= '1'; B <= '0'; A <= '0'; WAIT FOR 10 NS;
  D <= '1'; C <= '1'; B <= '0'; A <= '1'; WAIT FOR 10 NS;
  D <= '1'; C <= '1'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
  D <= '1'; C <= '1'; B <= '1'; A <= '1'; WAIT;
END PROCESS;

END A_T_CIRCUITO;
```

Seminario: Introducción al uso del software Vivado de Xilinx

Simulación del sistema

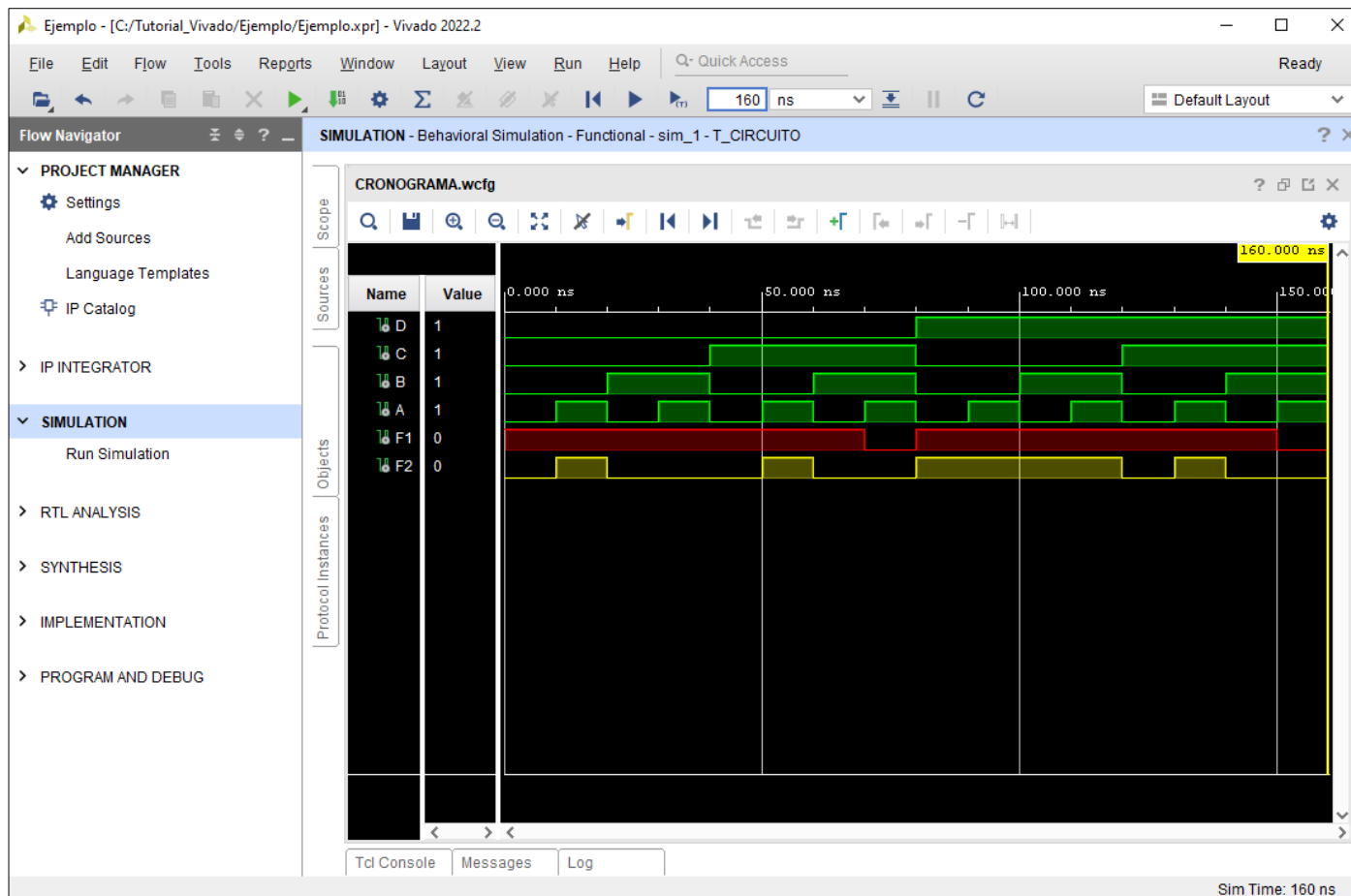
- Una vez editado el fichero, en la zona **Flow Navigator** de la pantalla principal de Vivado desplegamos la opción **Simulation**, seleccionamos **Run Simulation** y ejecutamos **Run behavioral Simulation** para realizar una simulación de comportamiento, como se muestra en la figura.



Seminario: Introducción al uso del software Vivado de Xilinx

Simulación del sistema

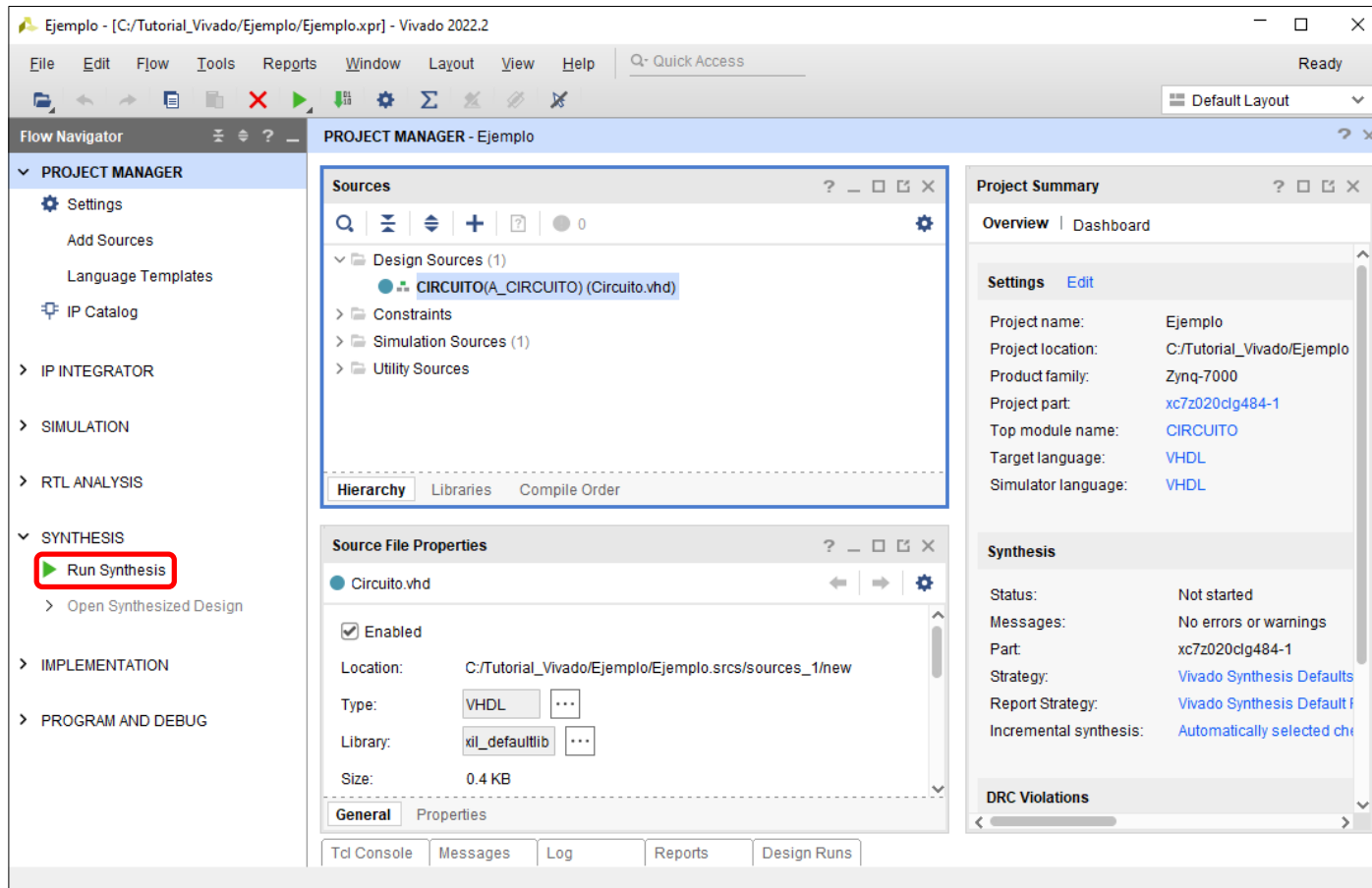
- Tras hacer esto, aparece una ventana que informa de que se está ejecutando la simulación y posteriormente se muestra un cronograma como el de la figura, donde podemos cambiar los colores de las señales, ajustar el tiempo de simulación, ampliar o reducir el intervalo de tiempo representado en la pantalla, etc.



Seminario: Introducción al uso del software Vivado de Xilinx

Síntesis del sistema

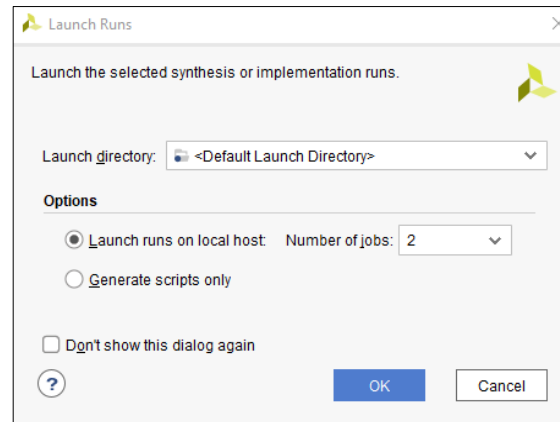
- Una vez comprobado el correcto funcionamiento del sistema procedemos a la síntesis del mismo, mediante la cual se genera un circuito físico cuyo comportamiento se corresponda con el código VHDL realizado previamente. Para ello, en la zona **Flow Navigator**, desplegamos la opción **Synthesis** y a continuación ejecutamos **Run Synthesis**, como se muestra en la figura.



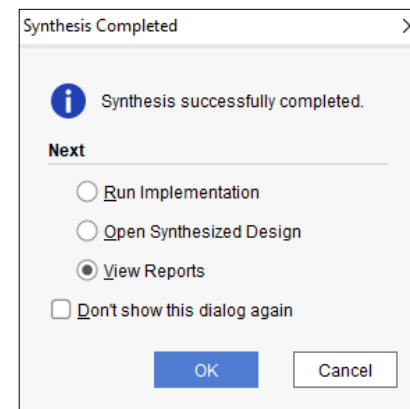
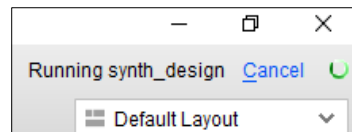
Seminario: Introducción al uso del software Vivado de Xilinx

Síntesis del sistema

- Tras ejecutar la opción **Run Synthesis** aparece la ventana de la siguiente figura, donde se nos solicita la confirmación antes de iniciar la síntesis del diseño.

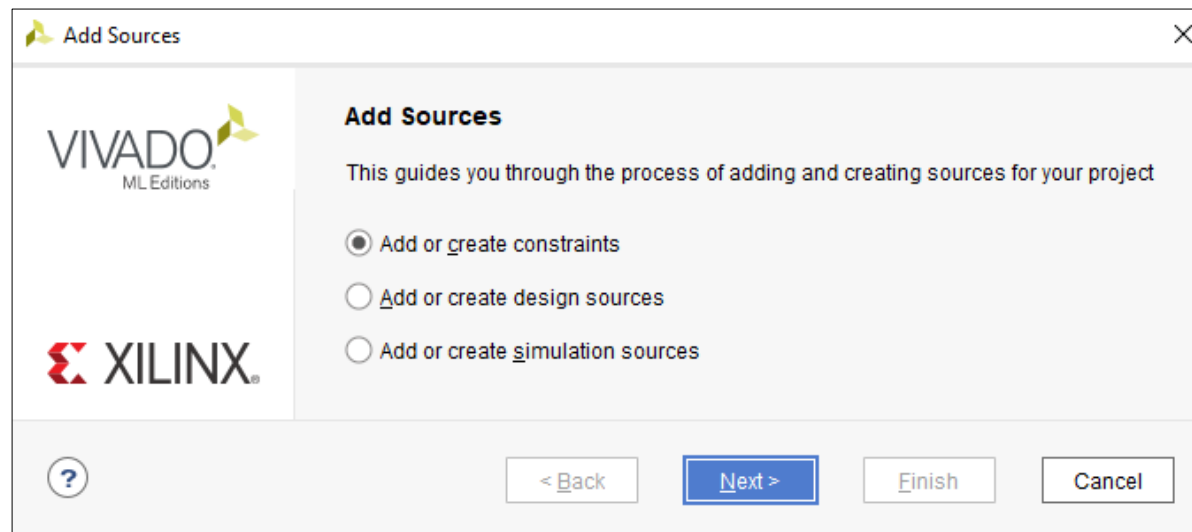


- Después de activar el botón **OK**, en la parte superior derecha de la pantalla podemos observar la evolución del proceso de síntesis (figura izquierda). Por último, si la síntesis concluye sin errores, el programa lo comunica mediante una ventana como la de la figura derecha.



Implementación del sistema

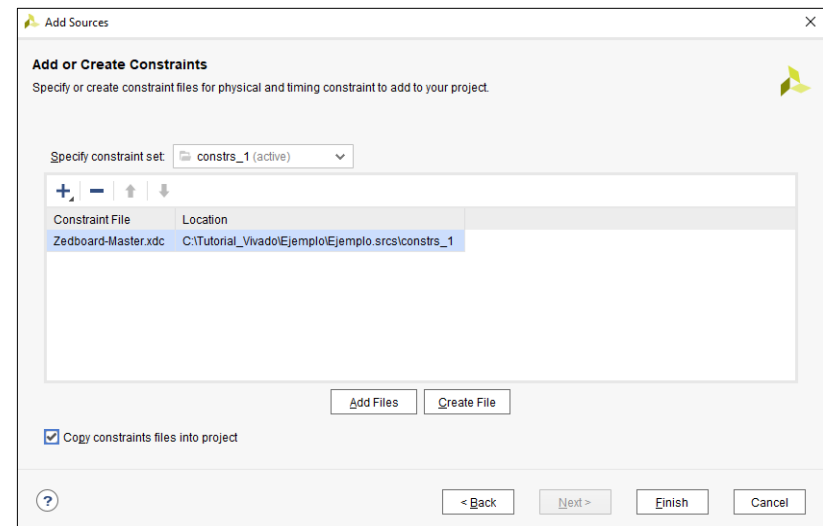
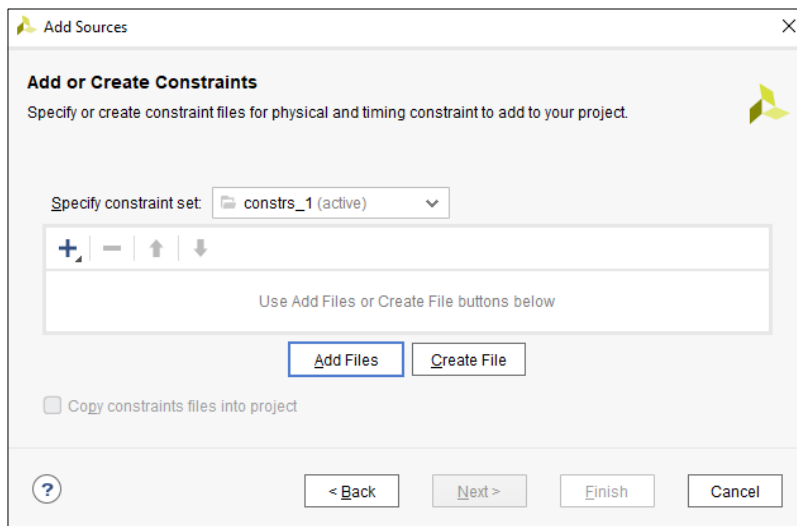
- La siguiente fase del proceso de diseño es la implementación del sistema. En esta fase se lleva a cabo la transformación del circuito modelado previamente para permitir su posterior implementación en un dispositivo FPGA integrado en una tarjeta de desarrollo.
- Para poder indicar a Vivado cuales son los terminales de la FPGA a los que deseamos conectar las entradas y salidas de nuestro circuito (D, C, B, A, F1 y F2), antes de ejecutar la implementación, debemos añadir al proyecto un **fichero de restricciones**, que es un fichero de texto que contiene información sobre los elementos de la tarjeta que podemos usar.
- Para añadir al proyecto el fichero de restricciones de la tarjeta ZedBoard proporcionado por el fabricante (***ZedBoard-Master.xdc***), seleccionamos **Add Sources** y a continuación, en la ventana que aparece, ejecutamos la opción **Add or create constraints**, como se muestra en la figura.



Seminario: Introducción al uso del software Vivado de Xilinx

Implementación del sistema

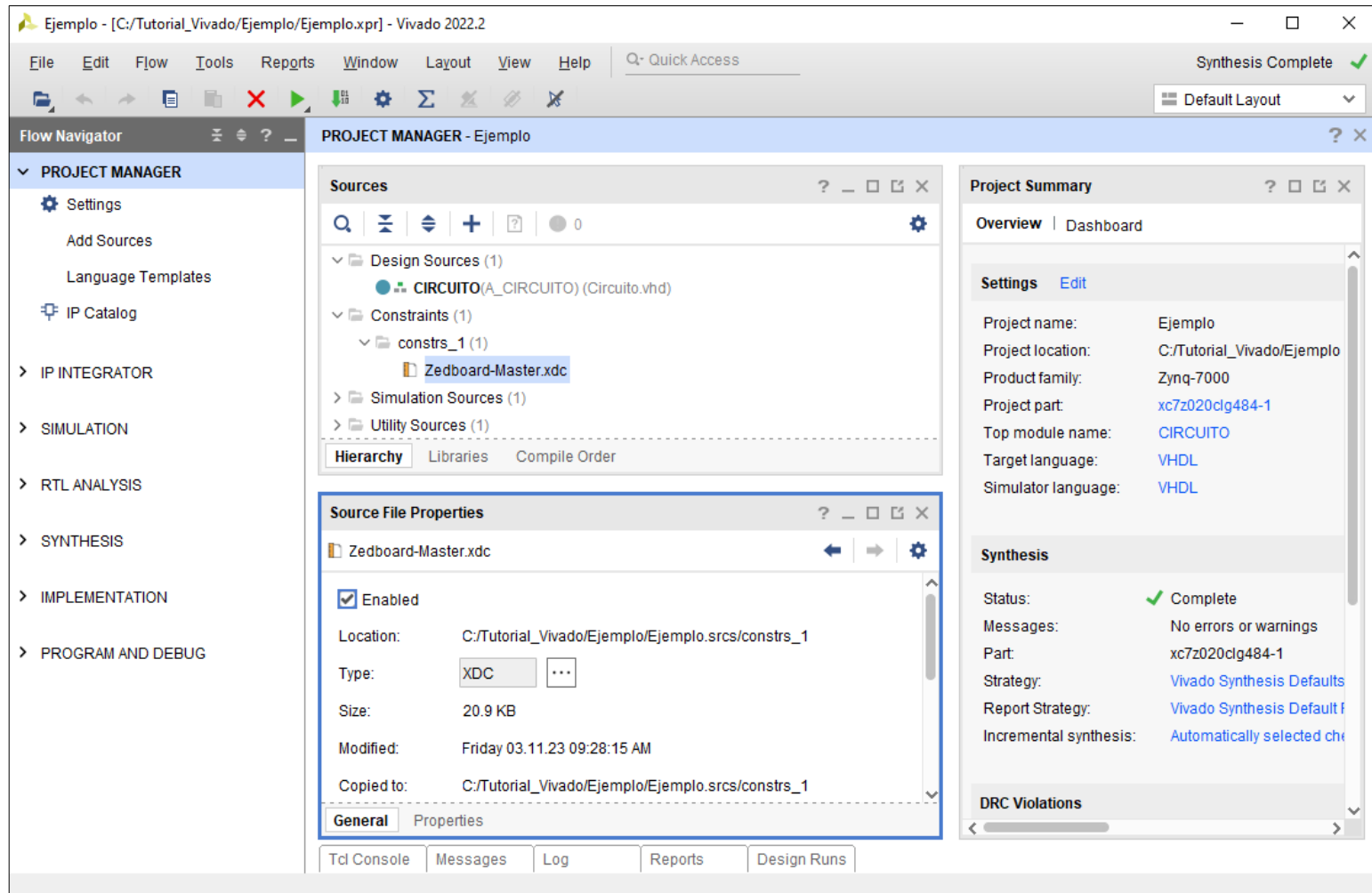
- Tras seleccionar la opción **Add o create constraints** y pulsar sobre **Next** aparece una ventana como la de la figura izquierda, en la que hacemos click sobre **Add Files**, ya que disponemos del fichero de restricciones de la tarjeta proporcionado por el fabricante.
- A continuación, seleccionamos el fichero de restricciones **Zedboard-Master.xdc** en la ubicación donde lo hayamos guardado previamente y pulsamos sobre **OK**, con que aparece una ventana como la de la figura derecha, donde se puede observar que ya se muestra el fichero de restricciones añadido al proyecto.
- Además, debemos seleccionar la opción **Copy constraints files into project**, para que Vivado asocie el fichero de restricciones al proyecto.



Seminario: Introducción al uso del software Vivado de Xilinx

Implementación del sistema

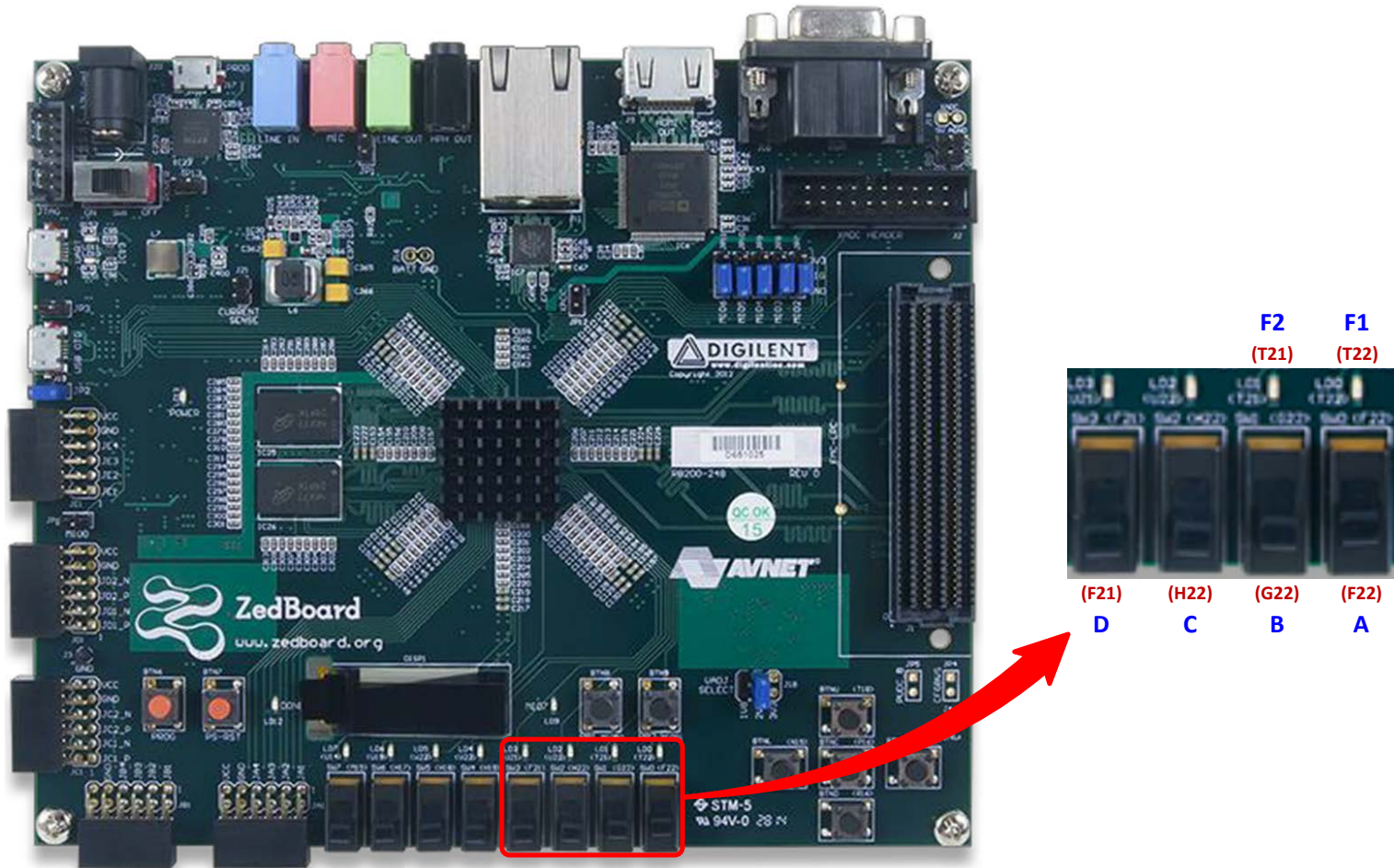
- Tras pulsar sobre **Finish**, observamos que en la ventana principal de Vivado ya aparece asociado al proyecto el fichero de restricciones añadido, como se muestra en la figura.



Seminario: Introducción al uso del software Vivado de Xilinx

Implementación del sistema

- Para este ejemplo, vamos a conectar los terminales de nuestro circuito tal como se muestra en la siguiente figura, donde se representan en color rojo los nombres de los pines de la FPGA a los que se encuentran conectados en la tarjeta ZedBoard los conmutadores y leds a emplear, y en color azul los nombres de los terminales del circuito que queremos asociar a estos elementos.



Implementación del sistema

- Para asociar las entradas y salidas del circuito a los terminales de la FPGA en la tarjeta de desarrollo editamos el fichero de restricciones haciendo doble click sobre su nombre, con lo que éste se abre en la parte derecha de la pantalla. Debemos mantener comentadas (con una # al principio) todas las líneas del mismo, excepto las que se indican a continuación.
- A las líneas correspondientes a los LEDs a emplear (**LD0** y **LD1**), les quitamos la # inicial y en la parte de `[get_ports {LDx}]`, sustituimos `{LD0}` por **F1** y `{LD1}` por **F2**, del siguiente modo:

User LEDs - Bank 33

```
set_property PACKAGE_PIN T22 [get_ports F1];
```

```
set_property PACKAGE_PIN T21 [get_ports F2];
```

- A las líneas de los conmutadores (**SW0**, **SW1**, **SW2** y **SW3**) les quitamos la # inicial y en la parte de `[get_ports {SWx}]`, sustituimos `{SW0}` por **A**, `{SW1}` por **B**, `{SW2}` por **C** y `{SW3}` por **D**:

User DIP Switches - Bank 35

```
set_property PACKAGE_PIN F22 [get_ports A];
```

```
set_property PACKAGE_PIN G22 [get_ports B];
```

```
set_property PACKAGE_PIN H22 [get_ports C];
```

```
set_property PACKAGE_PIN F21 [get_ports D];
```

- Por último, deberemos mantener sin comentar las cuatro líneas siguientes:

Note that the bank voltage for IO Bank 33 is fixed to 3.3V on ZedBoard.

```
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 33]];
```

Set the bank voltage for IO Bank 34 to 1.8V by default.

```
set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 34]];
```

Set the bank voltage for IO Bank 35 to 1.8V by default.

```
set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 35]];
```

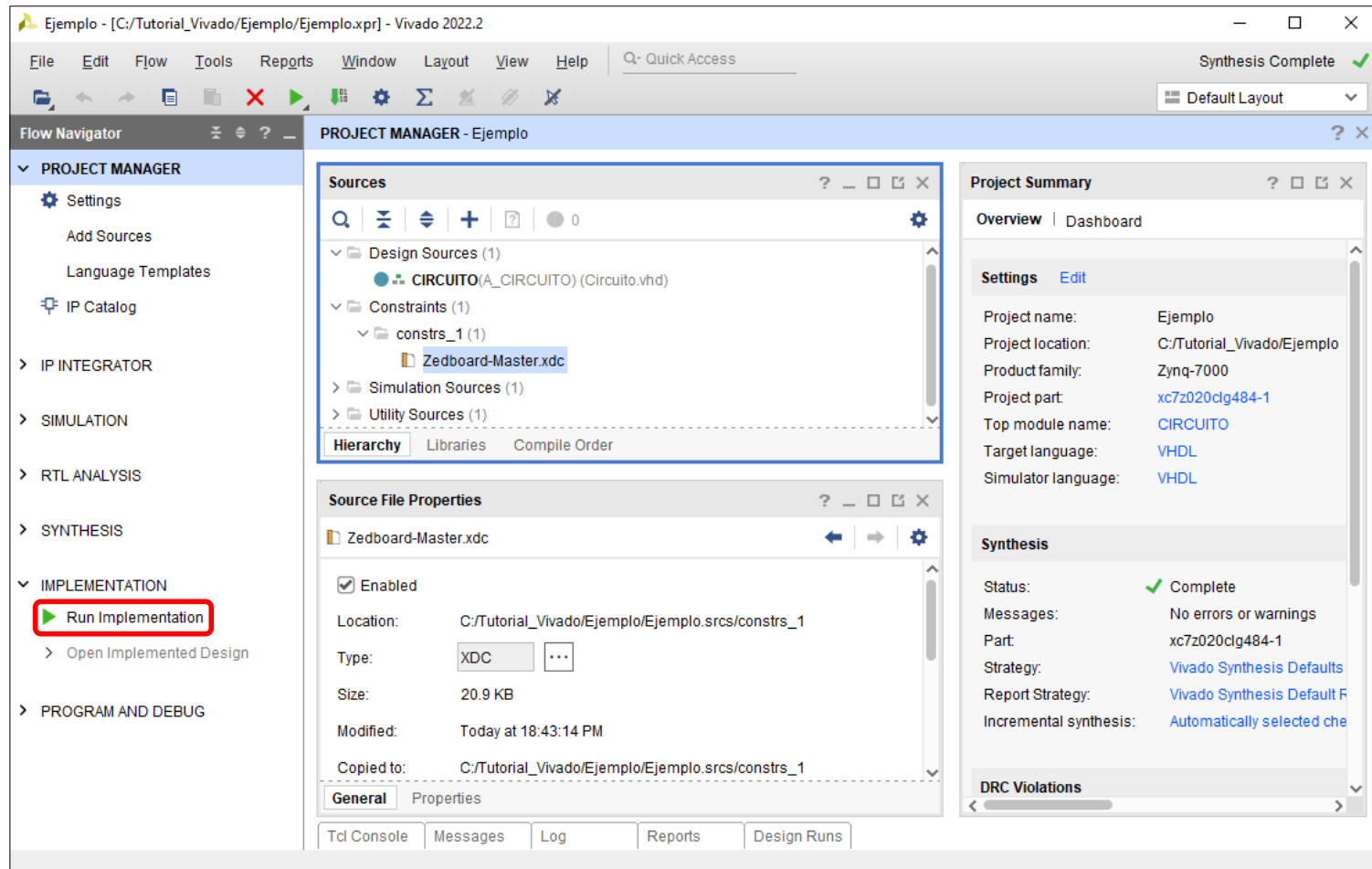
Note that the bank voltage for IO Bank 13 is fixed to 3.3V on ZedBoard.

```
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 13]];
```

Seminario: Introducción al uso del software Vivado de Xilinx

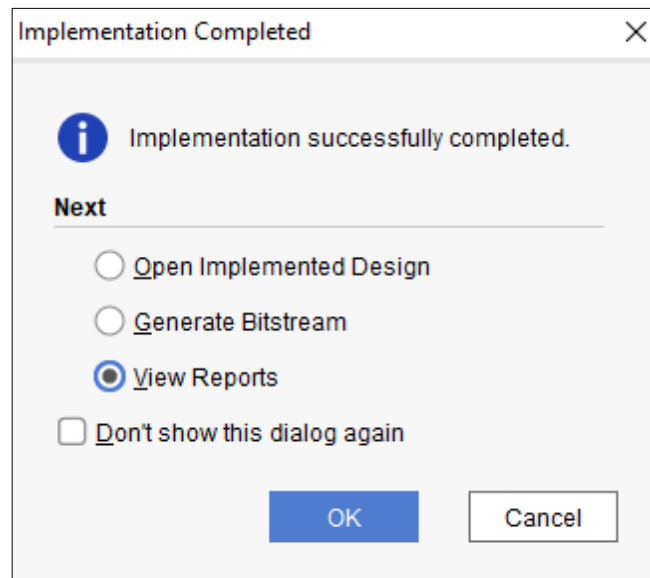
Implementación del sistema

- Una vez editado el fichero de restricciones, ejecutamos la implementación del sistema desplegando la opción **Implementation** en la zona **Flow Navigator** de la parte izquierda de la pantalla principal de Vivado y ejecutando a continuación **Run Implementation**.



Implementación del sistema

- Si la implementación del diseño concluye sin errores, el programa nos lo indica mediante una ventana emergente como la mostrada en la figura, que además nos permite abrir el diseño implementado, generar el Bitstream o visualizar los informes relativos al proceso realizado.

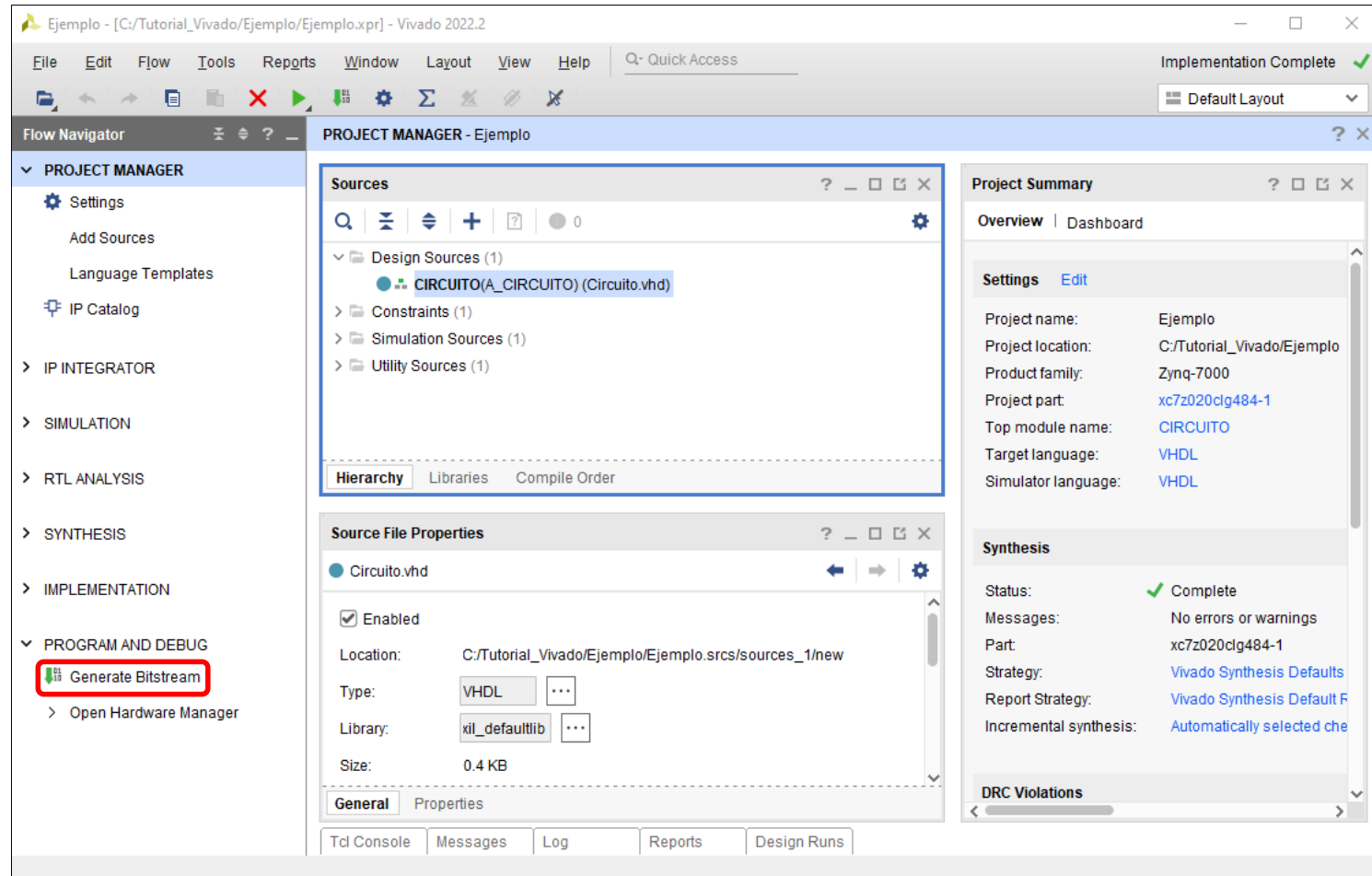


- En esta ventana, seleccionaremos *View Reports* y haremos click sobre *OK*.

Seminario: Introducción al uso del software Vivado de Xilinx

Programación de la tarjeta de desarrollo

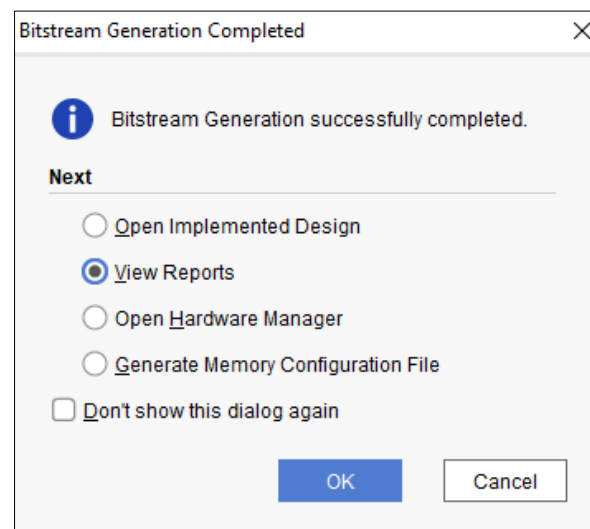
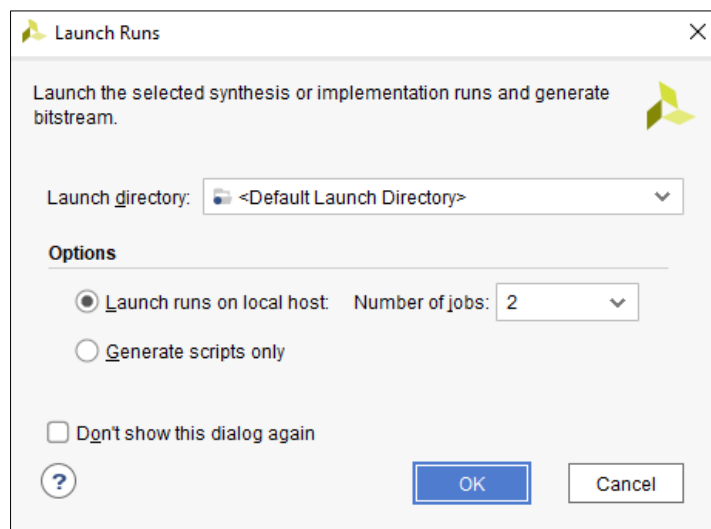
- Antes de programar la tarjeta de desarrollo debemos generar el **Bitstream** o fichero de programación. Para ello, una vez concluida con éxito la implementación del sistema, desplegamos la opción **Program and Debug** y a continuación ejecutamos **Generate Bitstream**.



Seminario: Introducción al uso del software Vivado de Xilinx

Programación de la tarjeta de desarrollo

- Tras ejecutar la opción **Generate Bitstream** aparece la ventana de la figura izquierda, donde se nos solicitará la confirmación antes de iniciar la generación del fichero.
- Tras validar con **OK** en esta ventana, después de la aparición de algunas ventanas intermedias similares a las observadas en procesos anteriores, el programa nos informa de que ha terminado la generación del Bitstream mediante una ventana como la de la figura derecha.

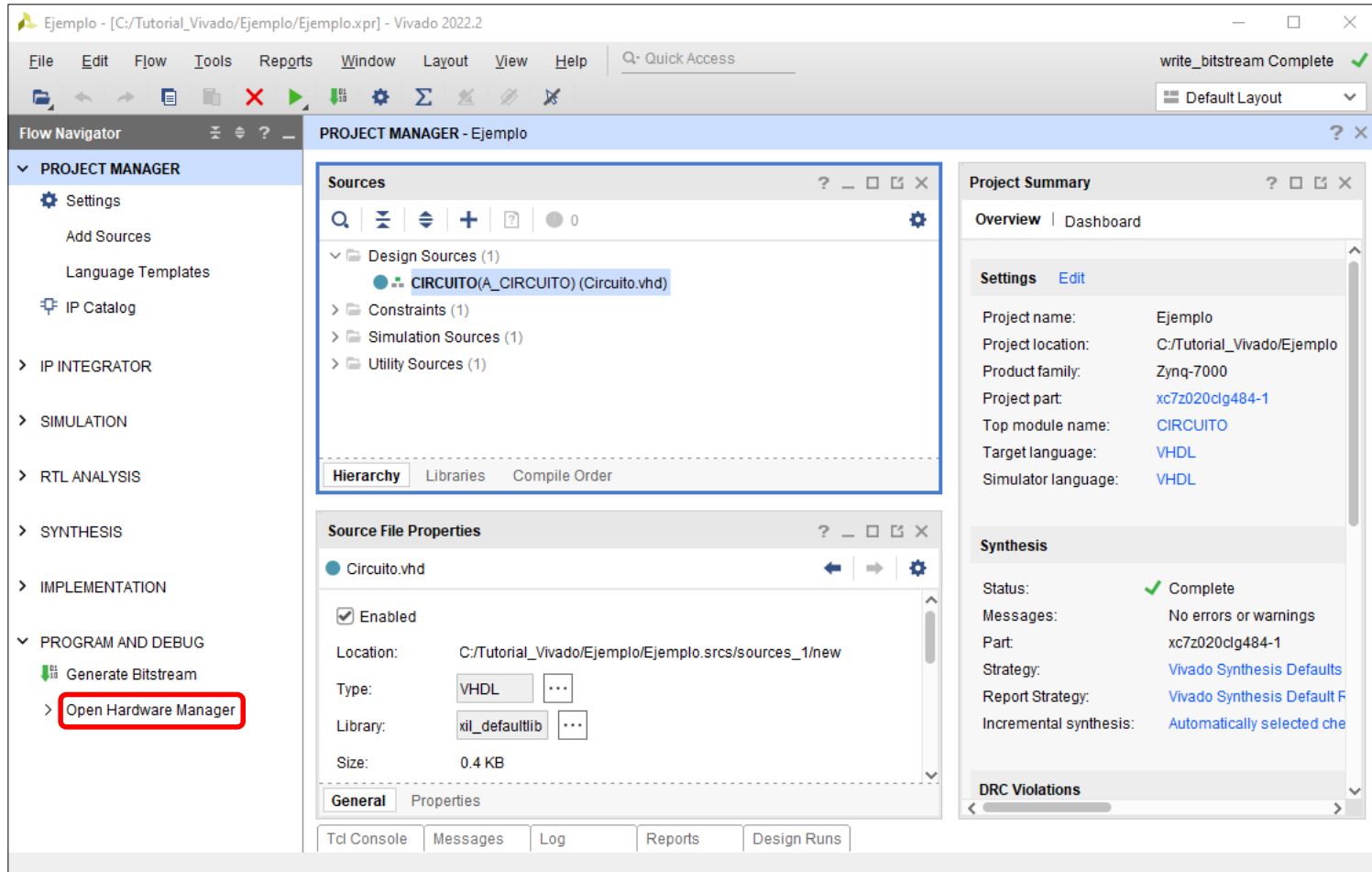


- Desde esta última ventana podremos abrir el diseño implementado, visualizar informes sobre el proceso realizado, abrir el gestor del hardware o generar un fichero de configuración para la memoria.
- En nuestro caso, seleccionamos la opción **View Reports** y validamos con OK.

Seminario: Introducción al uso del software Vivado de Xilinx

Programación de la tarjeta de desarrollo

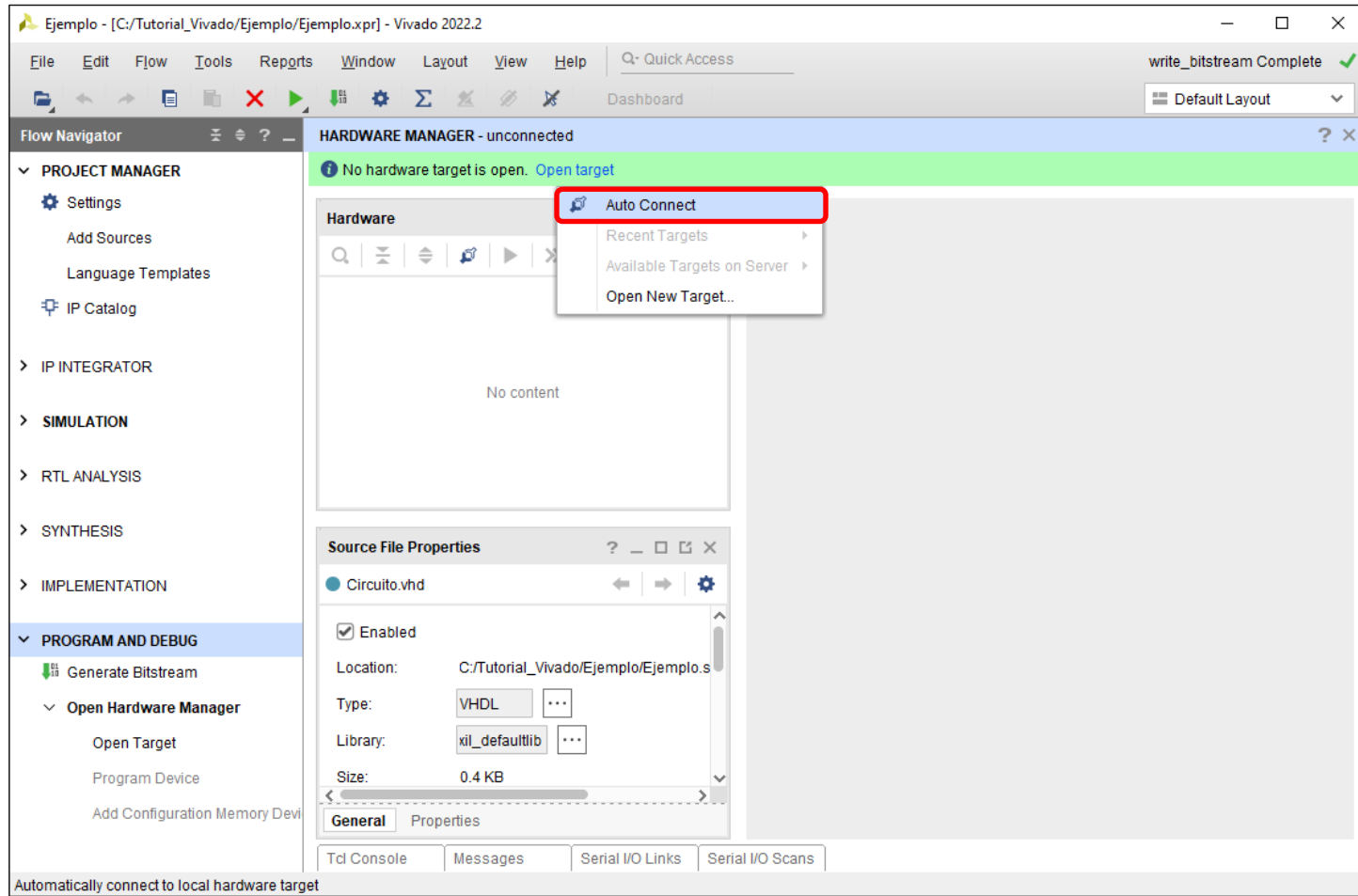
- Una vez creado el Bitstream, para programar la tarjeta de desarrollo conectamos ésta al ordenador a través del puerto USB, comprobamos que está encendida y a continuación ejecutamos la opción **Open Hardware Manager**, situada en la parte izquierda de la pantalla.



Seminario: Introducción al uso del software Vivado de Xilinx

Programación de la tarjeta de desarrollo

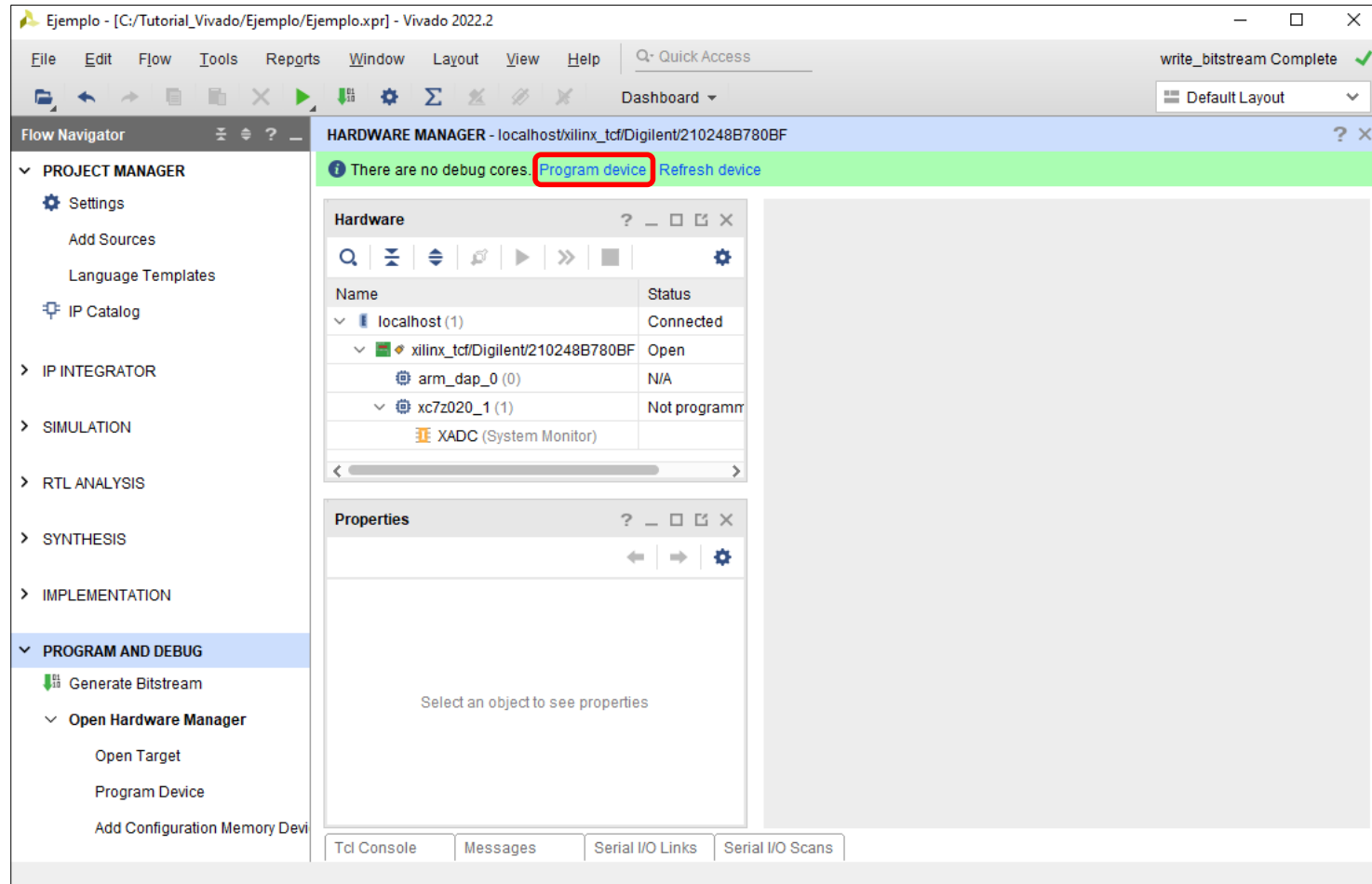
- A continuación, en la sección **Hardware Manager**, haremos click sobre **Open target** y seguidamente sobre **Auto Connect** para establecer la conexión con la tarjeta de desarrollo, como se muestra en la figura.



Seminario: Introducción al uso del software Vivado de Xilinx

Programación de la tarjeta de desarrollo

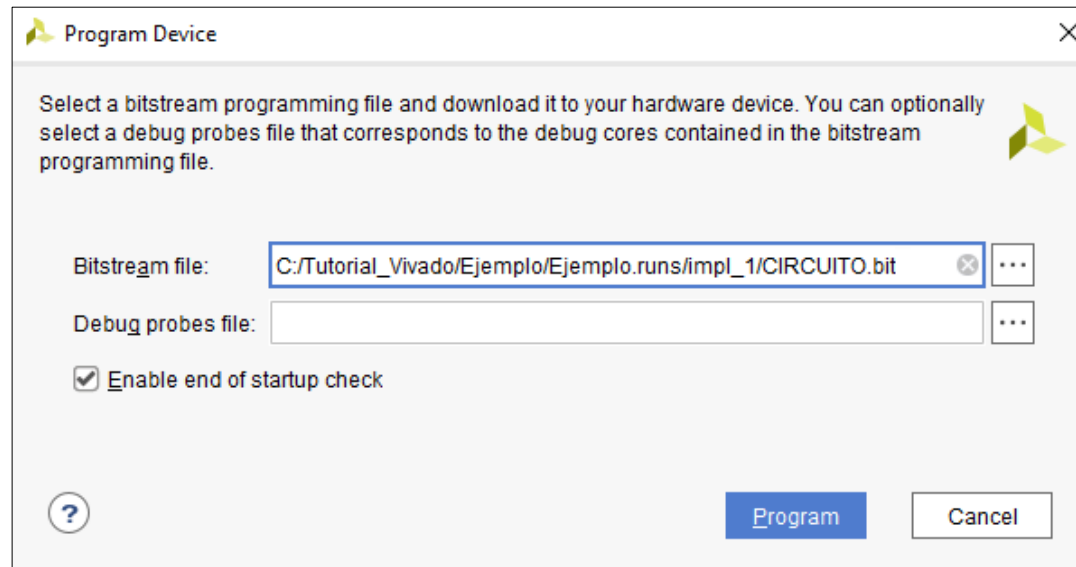
- Seguidamente, hacemos click sobre **Program Device**, como se muestra en la figura.



Seminario: Introducción al uso del software Vivado de Xilinx

Programación de la tarjeta de desarrollo

- Una vez hecho esto, aparece la ventana de la figura, donde el software nos indica cual el fichero de programación que va a usar para programar la tarjeta de desarrollo, en este caso **Circuito.bit**.



- Por último, tras hacer click sobre **Program** se lleva a cabo la programación de la tarjeta.
- Una vez finalizada la programación de la tarjeta de desarrollo se nos avisa mediante el encendido del **LED DONE** de dicha tarjeta. A continuación, manipulamos los conmutadores de la misma para aplicar al circuito las diferentes combinaciones de entrada y observamos el encendido o no encendido de los LEDs para comprobar que los valores de las salidas son los correctos para cada una de ellas.